

Towards a RISC-V Firmware Scheduler for
Adaptive Wireline SerDes Equalization

by

Matthew Oliveira, Gustavo Nakashita,

Satvick Acharya, and Andreas Conostas-Malevanets

B.A.Sc. Capstone in Electrical and Computer Engineering

Table of Contents

Table of Contents	2
List of Abbreviations	4
Chapter 1. Framing - Challenges of Equalization for Wireline SerDes	5
1.1 Introduction to the Context	5
1.1.1 Introduction to SerDes	7
1.1.2 Introduction to Wireline	8
1.1.3 Introduction to SerDes Equalization	9
1.1.4 The Need for a Firmware Scheduler for Link Training and Monitoring	10
1.1.5 Problem Space and Stakeholders	12
1.1.5.1a Stakeholder 1: Marvell Technology	12
1.1.5.1b Marvell's Stakeholders	13
1.1.5.2 Stakeholder 2: Professor Ali Sheikholeslami	14
1.1.5.3 Stakeholder 3: The Capstone Team	14
1.1.5.4 Stakeholder Relationships and Resulting Design Need	15
1.1.5.5 Current (Legacy) Solution and Gap	15
1.2 Value Proposition and General Approach	16
1.3 Engineering Requirements	17
1.3.1 The Need for a RISC-V Scheduler	22
1.4 Broader Implications and Ethical and Legal Considerations	22
1.4.1 Implications for Industry, Academia, and Society	22
1.4.2 Environmental Considerations	23
1.4.3 Safety, Reliability, and Ethical Considerations	24
1.4.4 Legal, Standards, and Institutional Considerations	24
1.5 Project Timeline	25
Chapter 2. Conceptual Design	28
2.1 Scheduler design considerations	28
2.1.1 Available Open-Source Schedulers	29
2.1.2 Hardcoded SerDes Scheduler	31
2.1.2.1 Hardcoded SerDes Scheduler with Block Coordinate Descent	31
2.1.3 Customized Programmable Scheduler	32
2.1.4 Non-Scheduler Alternatives	32
2.2 Selected Conceptual Design: Customized Programmable Scheduler	33
2.2.1 Justification for Selected Conceptual Design	33
2.2.2 Interactions with Larger Operational Environment	34
2.2.3 Potential Behaviors	35
2.2.4 User Interactions	38
2.3 Proof-of-Concept Considerations	38
Chapter 3. Prototyping and Evaluation	41

3.1 System Architecture	41
3.2 Implementation Summary	44
3.2.1 MATLAB SerDes Testbench	45
3.2.1.1 Troubleshooting MATLAB Implementation	48
3.2.1.2 Removing TX FFE	48
3.2.2 QEMU Scheduler	48
3.2.3 RISC-V Softcore Implementation	49
3.2.3.1 Softcore Processor Selection	49
3.2.3.2 FPGA Platform and Clocking	51
3.2.3.3 Processor Configuration	51
3.2.3.4 Memory Architecture	51
3.2.3.5 Timer, Peripherals, and Debugging	52
3.2.3.6 Firmware Deployment and Portability	52
3.2.4 PC-FPGA Communication Interface	53
3.2.5 Scheduler Software Performance Optimization	54
3.3 Prototype Evaluation Procedure	58
3.3.1 QEMU Evaluation Procedure	58
3.3.2 Limitations of QEMU Evaluation	59
3.3.3 FPGA Evaluation Procedure	61
3.3.4 Limitations of the FPGA Evaluation	62
3.4 Scheduler Performance Benchmarking	63
3.4.1 QEMU Benchmarking	63
3.4.2 FPGA Benchmarking	64
3.5 Evaluating Scheduler for SerDes Application	66
3.5.1 System Overview	66
3.5.2 Channel Simulation	66
3.5.3 Serial Communication Protocol	67
3.5.4 Multi-Lane Training and Scheduling	67
3.5.5 Graphical User Interface	68
3.5.6 PC-Based SerDes Link Training Interface Example Operation	69
Chapter 4. Discussion Conclusions and Future Considerations	71
Chapter 5. Project Team	72
References	74
Appendix A: Standard Equalization Techniques	77
Technical Introduction to SerDes Equalization	77
Feed-Forward Equalization (FFE)	78
Continuous-time linear equalization CTLE)	79
Decision Feedback Equalization (DFE)	79
Algorithms for Equalization Training	79

List of Abbreviations

ABI Application Binary Interface	PCIe Peripheral Component Interconnect Express
ADC Analog-to-Digital Converter	PLL Phase-Locked Loop
API Application Programming Interface	PRBS Pseudo-Random Bit Sequence
BCD Block-Coordinate Descent	RAM Random Access Memory
CDR Clock-Data Recovery	RLS Recursive Least Squares
CLINT Core Local Interruptor	RX Receiver
CPI Cycles per Instruction	SerDes Serializer-Deserializer
CPU Central Processing Unit	SoC System-on-Chip
CSR Control and Status Register	SRAM Static RAM
CTLE Continuous-Time Linear Equalizer	TX Transmitter
DFE Decision-Feedback Equalizer	UART Universal Asynchronous Receiver-Transmitter
DMEM Data Memory	USB Universal Serial Bus
DRAM Dynamic RAM	XBUS External Bus Interface
FFE Feed-Forward Equalizer	
FF Flip-Flop	
FFT Fast Fourier Transform	
FIFO First-In, First-Out	
FPGA Field-Programmable Gate Array	
GUI Graphical User Interface	
IFFT Inverse Fast Fourier Transform	
IMEM Instruction Memory	
ISI Inter-Symbol Interference	
ISR Interrupt Service Routine	
LMS Least-Mean Squares	
LUT Look-Up Table	
MCU Microcontroller Unit	
MSE Mean-Squared Error	
OCD On-Chip Debugger	
PAM Pulse-Amplitude Modulation	
PC Personal Computer	

Chapter 1. Framing - Challenges of Equalization for Wireline SerDes

1.1 Introduction to the Context

Modern computing workloads, particularly those driven by artificial intelligence and cloud-scale data processing, increasingly depend on large networks of parallel hardware [X1]. In data centres, this computation is distributed across many processors, accelerators, graphics cards, and memory devices. Therefore, system performance depends not only on raw compute capability but also on the ability to move data reliably between chips at high rates, with the latter increasingly becoming the primary bottleneck [X, X]. As a result, high-speed wireline communication has become a central design concern for semiconductor companies and their customers. Standards such as PCIe 6.0 reflect this trend by requiring substantially higher link data rates than previous generations [X2].

This communication occurs across multiple physical scales within modern computing infrastructure, as illustrated in Figure 1. At the macro level, servers within a data centre exchange data over long-reach interconnects (e.g., Ethernet), enabling distributed computation across racks and clusters. At a finer scale, within a single server, high-speed electrical links connect processors, accelerators, and memory devices across printed circuit boards and packages. It is at this intra-server, chip-to-chip level that wireline serializer-deserializer (SerDes) links operate, forming the focus of this work.

This creates an opportunity and a challenge for companies such as Marvell developing SerDes products. Their customers: hyperscalers, data-centre operators, and chip designers, need faster and more reliable interconnect chips to support growing compute demand [X]. Figure 1 illustrates this dynamic, where different members of the semiconductor industry offer compute and connectivity for hyper-scalers. However, achieving higher data rates requires operation at higher frequencies, where wireline channels suffer more severe signal degradation [X]. Effects such as attenuation, reflections, intersymbol interference, jitter, and noise make it harder for the receiver to correctly recover transmitted data. Without adequate compensation, these effects can reduce link reliability, increase errors and retransmissions, and ultimately limit system performance [X].

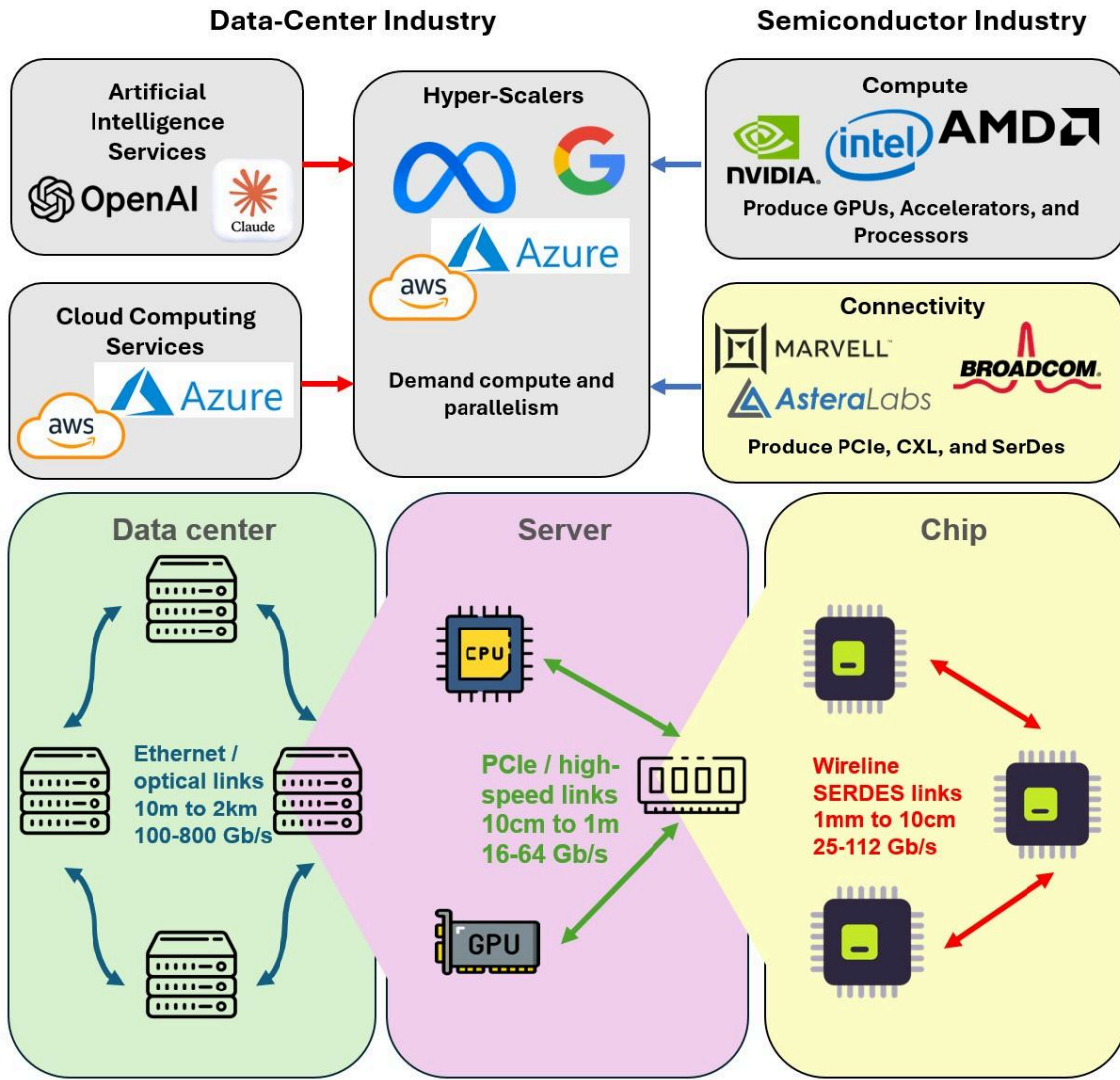


Figure 1. Overview of interaction between the semiconductor industry and the hyper-scaler industry. Hyper-scaler compute and interconnectivity requirements dictate the design decisions for players in the semiconductor industry [X2]–[X4]. This project focuses on wireline SerDes communication.

To address this, modern SerDes links use equalization techniques that adapt the transmitted and received signals to the characteristics of a particular channel. These techniques require link-training algorithms that tune parameters such as equalizer tap weights, gain characteristics, and filter settings [X]. In commercial systems, this training is typically coordinated by embedded firmware running close to the SerDes hardware [X, X, X].

Therefore, the quality of the firmware infrastructure matters: it must schedule training and monitoring tasks quickly, deterministically, and with low memory overhead while supporting many independent lanes whose channel conditions may vary.

Data-centre customers need high-throughput links with minimal downtime. SerDes vendors such as Marvell need firmware infrastructure that can support increasingly complex link-training procedures without excessive area, power, memory or latency costs. Academic stakeholders, including Professor Sheikholeslami's research group, need more open and reproducible methods for studying link-training algorithms and firmware-level implementation choices. This project is situated at the intersection of these needs; it investigates how a lightweight RISC-V based firmware scheduler can support SerDes equalization tasks in a way that is efficient, deterministic, and suitable for future link-training research and development.

1.1.1 Introduction to SerDes

Serializer-deserializer (SerDes) circuits are high-speed communication blocks that first convert parallel digital data into a serial stream for transmission across a small number of physical high-speed channels, then afterwards reconstruct the original parallel data at the receiver as shown in Figure 2. Instead of sending many bits simultaneously across separate wires, a SerDes link sends data sequentially at a much higher rate across one serial lane or a small group of lanes. This reduces pin count, routing complexity, package area and interconnect cost, which makes SerDes especially valuable in systems where many chips must communicate within tight physical and power constraints [X5].

SerDes links are widely used in high-performance computing and data-centre hardware, including chip-to-chip communication, die-to-die interfaces, board-level interconnects, PCIe links, and Ethernet systems [X4]. For stakeholders such as semiconductor vendors and their data-centre customers, SerDes enables the high-bandwidth connectivity needed to move data between processors, accelerators, memory, and networking devices. However, as the data is concentrated into fewer physical channels, each channel must operate at very high symbol rates in the hundreds of gigasymbols per second. At these speeds, the electrical behavior of the wireline channel becomes a severe limiting factor, and the received signal can be distorted by loss, reflections, noise, and timing uncertainty which must be accounted for by the transceiver circuit.

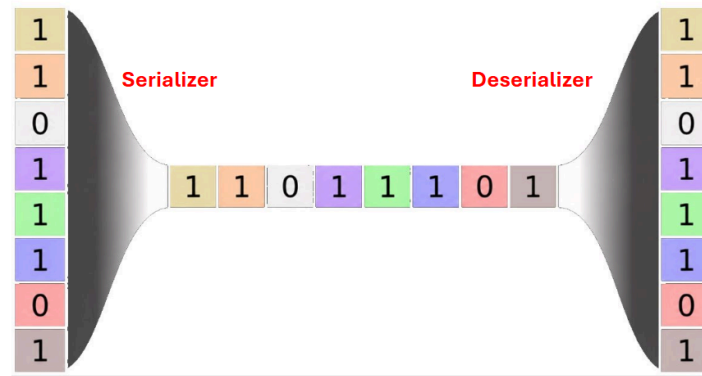


Figure 2: Representation of the function of a SerDes link. Adapted from [X6].

1.1.2 Introduction to Wireline

High-speed wireline links transmit digital information as electrical waveforms through physical interconnects such as PCB traces, cables, connectors and package routes. At lower data rates, these interconnects can often be treated approximately as ideal conductors. However, as symbol rates increase, the physical properties of the channel become significant. Frequency-dependent loss from reflections, impedance, discontinuities, crosstalk, and noise all change the shape of the transmitted waveform before it reaches the receiver.

One of the main consequences of this distortion is intersymbol interference (ISI), where energy from one transmitted bit or symbol spreads into neighboring symbols, potentially causing error in the received circuit if the ISI exceeds the thresholds set in the receive circuit's analog slicer. Rather than observing a clean sequence of distinct voltage levels, the receiver observes a degraded waveform in which symbol boundaries are harder to identify. As illustrated in Figure 2, this can cause the receiver to make an incorrect decision about the transmitted data leading to an error. If these errors occur frequently, the link breaks down and can no longer support reliable communication between chips.

The channel behavior creates a central challenge for high-speed SerDes systems: increasing the data rate also makes the physical link harder to interpret correctly. Therefore, reliable high-speed wireline communication requires techniques that compensate for channel distortion and allow the receiver to recover the intended data with a sufficiently low error rate.

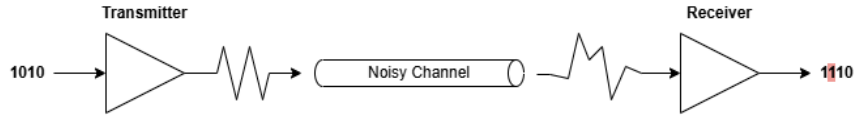


Figure 2. Simplified example of communication across a noisy channel. Without equalization, the noisy channel produces an error in the received signal.

1.1.3 Introduction to SerDes Equalization

Equalization is the primary technique used in SerDes systems to compensate for distortion introduced by the wireline channel. Rather than expecting the physical interconnect to preserve the transmitted waveform, equalization deliberately reshapes the signal before transmission, after reception, or both as shown in Figure 3. The goal is not to eliminate all noise, but to reduce the effects of the channel loss and intersymbol interference enough that the receiver can reliably distinguish the transmitted symbols.

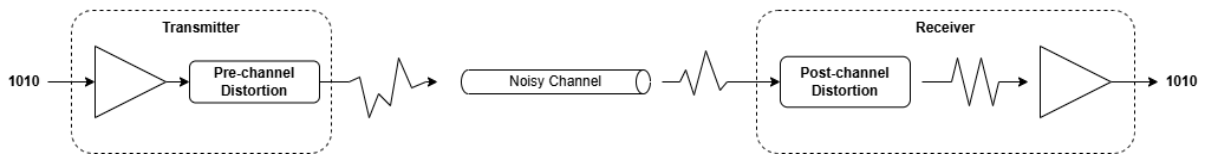


Figure 3. Simplified, idealized example of communication with equalization blocks introduced. Both the pre-channel and post-channel distortion blocks are able to counteract the noise and produce a signal at the receiver end that correctly identifies the bits.

Modern SerDes links commonly use a combination of equalization blocks, including Feed-Forward Equalizers (FFE), Continuous-Time Linear Equalizers (CTLE), and Decision-Feedback Equalizers (DFE) [X4]. Transmitter FFE and receiver FFE stages can reshape the signal using weighted tap values, CTLE can boost attenuated high-frequency components before digitization, and DFE can cancel interference from previously detected symbols. Further technical details on these equalization methods are provided in Appendix A.

However, equalization is only effective when its parameters are tuned to the specific channel being used. Each lane may experience different loss, reflection, and noise conditions because of differences in routing, packaging, connectors, or operating environment, most notably temperature variation. As a result, a SerDes system cannot rely on one fixed equalizer setting

for all cases. It must perform link training, where a firmware or control logic adjusts equalization parameters until the received signal quality is acceptable. Additionally, as conditions such as temperature and wear change the characteristics of the wireline, repeated re-training is needed to ensure the channel remains operational. This need for repeated, lane-specific tuning motivates the firmware scheduling problem addressed in this project.

1.1.4 The Need for a Firmware Scheduler for Link Training and Monitoring

The equalization blocks described above are not fixed circuits with one universal configuration. Their performance depends on several tuneable parameters, including FFE and DFE tap weights, CTLE pole and zero locations, and gain settings. The best values for these parameters depend on the specific channel, data rate, the lane used, and operating conditions, which may vary over time. As a result, SerDes systems require a link-training process that measures and adjusts equalization settings until the received signal can be recovered reliably. Even after initial training, performance must be monitored and settings adjusted as conditions change.

In modern SerDes implementations, all equalization, link training, and monitoring functions are executed on a single embedded microcontroller as shown in Figure 4 to minimize area cost. This forces all logically concurrent adaptation and monitoring processes to compete for a single processing resource. This architectural constraint makes execution inherently sequential, despite the need to service multiple time-sensitive tasks. Without a scheduler, execution order is dictated by ad hoc polling structures or interrupt arrival patterns, leading to uncontrolled timing, variable latency, and potential task starvation. Some routines may execute too frequently while others are delayed, and there is no guarantee that critical adaptation loops receive processor time at the required intervals. As a result, the system cannot ensure consistent update rates or bounded latency—both of which are essential for reliable link operation.

These timing inconsistencies directly impact the stability of the equalization process. Adaptive algorithms rely on regular, well-defined update intervals to converge. If updates occur irregularly, the adaptation process can become unstable, leading to slow convergence, or complete failure to converge, particularly in high-speed channels with tight noise and margin constraints. In addition, the equalization blocks embedded within the receiver are not

independent, but form a set of interdependent control loops. Adjustments in one stage (i.e. CTLE gain) directly alter the operating conditions of subsequent stages (i.e. DFE tap adaptation). This coupling means that uncontrolled or poorly timed execution can cause loops to operate on inconsistent system states, further exacerbating instability and producing non-deterministic behavior.

Finally, SerDes systems often include shared analog resources, such as PLLs and clock distribution networks, which may be used across multiple lanes [X7], [X8]. These resources impose strict coordination constraints, as their configuration directly determines the timing quality of the transmitted and recovered data. For example, re-tuning a PLL during link training may involve modifying loop parameters, divider settings, or initiating calibration cycles that temporarily disturb the output clock or force the PLL to relock. If another lane is simultaneously relying on this clock for data recovery, such disturbances can corrupt otherwise valid data transmission. This creates a fundamental conflict between lanes undergoing training and those in normal operation. Without controlled access, one lane's attempt to optimize its link can degrade or interrupt another's. A scheduler is therefore required to serialize and gate access to shared analog resources, ensuring that reconfiguration occurs only when it is safe and does not violate system-level timing constraints.

A firmware scheduler is therefore required to impose deterministic, ordered, and conflict-free execution. It ensures that all tasks sharing a single microcontroller are serviced in a controlled manner, maintains consistent timing for stable adaptation, enforces proper sequencing between interdependent loops, and arbitrates access to shared hardware resources. Without such coordination, reliable link training and continuous adaptation in high-speed SerDes systems would not be achievable.

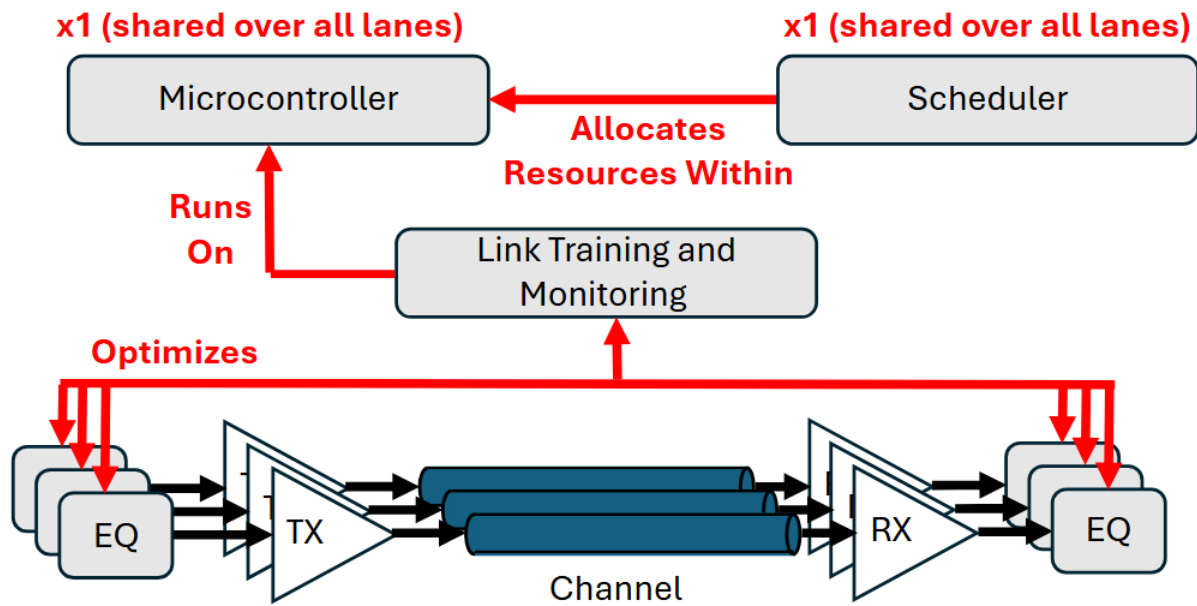


Figure 4: A firmware scheduler interfacing with several different SerDes links. The training algorithm observes channel outputs against a known bit sequence, and updates taps to recover the known sequence. The scheduler allows the scheduling algorithm to run on several lanes simultaneously.

1.1.5 Problem Space and Stakeholders

Commercial SerDes link-training algorithms are typically proprietary, and the firmware infrastructure used to run them is rarely discussed in public literature. This creates two connected problems: first, industry teams must continue evolving firmware for increasingly complex high-speed links while working within strict limits on timing, memory, and hardware resources; second, academic researchers have limited access to realistic implementation details, making it difficult to study link-training algorithms as complete hardware-software systems. This project addressed both sides of that problem by developing and evaluating a lightweight RISC-V-based scheduler for SerDes equalization tasks.

1.1.5.1a Stakeholder 1: Marvell Technology

Marvell Technology is the primary industrial stakeholder for this project. Marvell is a semiconductor company that develops networking switches, data-centre interconnects, and other high-speed communication hardware. In these systems, SerDes links must be trained

and monitored so that the transmitter and receiver equalization settings remain appropriate for the physical channel.

Marvell currently uses a legacy microcontroller-based firmware solution on a 8051 microcontroller to conduct SerDes link training and monitoring. While this approach is functional, it becomes increasingly limiting as SerDes systems scale to higher data rates, larger numbers of lanes, and more complex equalization procedures. Each additional lane or training stage increases the amount of firmware state that must be tracked, the number of tasks that must be scheduled, and the timing pressure on the controller. A legacy firmware approach that works for simpler link-management tasks may therefore become harder to extend without increasing memory usage, increasing latency, or making the firmware less predictable.

Portability is also a concern. A firmware solution that is tightly coupled to a specific legacy microcontroller or toolchain can be difficult to reuse across future SerDes products or adapt to new hardware platforms. This limits long-term maintainability and makes it harder to evolve the firmware architecture as training algorithms change. In contrast, RISC-V's open and modular instruction set architecture provides a more flexible platform for exploring custom embedded firmware solutions without relying on a proprietary processor architecture.

1.1.5.1b Marvell's Stakeholders

Marvell's own stakeholders include data-centre operators, cloud providers, system vendors, and companies developing high-performance computing and AI hardware. These downstream users may not directly consider the details of SerDes equalization, but they are affected by the reliability and performance of the links that equalization enables. If link training is slow, unstable, or difficult to scale, the resulting system may experience increased link downtime, reduced throughput, or greater design complexity. These system-level demands are translated by Marvell into firmware-level requirements, including low memory usage, fast task switching, low interrupt latency, and deterministic operation.

However, SerDes firmware is typically deployed without a full operating system, so this transition cannot simply rely on an existing general-purpose scheduler. The scheduler must be lightweight, deterministic, and suitable for strict embedded constraints. Marvell's main interest in this project was therefore to evaluate whether a RISC-V-based scheduler could

support SerDes link-training tasks while minimizing memory usage, reducing context-switch and interrupt latency, and preserving predictable execution timing.

1.1.5.2 Stakeholder 2: Professor Ali Sheikholeslami

Professor Ali Sheikholeslami is the primary academic stakeholder for this project. As a researcher in wireline communication systems at the University of Toronto, his interest is not limited to a specific commercial firmware platform. Instead, his main need is for a public and reproducible way to study how SerDes link-training algorithms can be implemented under realistic embedded firmware constraints.

Although equalization circuits and signal-processing algorithms are widely discussed in academic literature, the firmware systems that coordinate real link-training procedures are often proprietary and the hardware systems used to prototype and test link training across multiple lanes are prohibitively expensive. This makes it difficult for researchers and future student teams to study the interaction between the training algorithm, the scheduler, and the hardware platform. In particular, there is limited public guidance on how a link-training algorithm should be divided into scheduled tasks, how multiple lanes should be managed, and how timing and memory constraints affect the feasibility of a training procedure.

This project addressed that need by developing and validating a proof-of-concept scheduler that could execute SerDes-inspired link-training tasks on a modern embedded architecture. Paired with a simplified SerDes equalization model, the scheduler provided a concrete platform for testing whether a training algorithm could converge while operating within firmware constraints such as low memory usage, deterministic timing, and fast task switching. For Professor Sheikholeslami, the value of the project was therefore not only the scheduler itself, but also the creation of an open experimental framework that future researchers could use to study algorithm-scheduler co-design in high-speed wireline systems.

1.1.5.3 Stakeholder 3: The Capstone Team

The capstone team acted as both the design team and a stakeholder in the learning and implementation process. The team's objective was to translate high-level industrial and academic needs into a concrete proof-of-concept system: a lightweight scheduler implemented on a RISC-V platform and evaluated using SerDes-inspired equalization tasks.

This required balancing firmware performance, hardware constraints, algorithm behavior, and project scope.

The team also had an educational stake in the project. By working at the intersection of SerDes equalization, embedded firmware, and computer architecture, the team gained experience with a realistic engineering problem whose constraints are shaped by both commercial deployment and academic reproducibility. The final outcome was therefore intended to serve not only Marvell and Professor Sheikholeslami, but also future design teams seeking an open starting point for SerDes firmware experimentation.

1.1.5.4 Stakeholder Relationships and Resulting Design Need

The stakeholder relationships can be understood as a chain of requirements. End users of high-speed computing and networking systems demand reliable, high-throughput communication. These expectations place pressure on SerDes vendors such as Marvell to reduce link downtime and support increasingly complex equalization procedures. Marvell translates those system-level needs into firmware-level constraints, including low memory usage, deterministic timing, and fast task switching. Professor Sheikholeslami and the academic research community are interested in making these implementation challenges visible and reproducible, while the capstone team was responsible for producing a proof-of-concept system that addressed both the industrial and academic sides of the problem.

Together, these stakeholder needs motivated the central design opportunity: to develop a lightweight, deterministic, and portable scheduler for SerDes link-training tasks, implemented on an open RISC-V platform and validated using a simplified but representative equalization workload. This framing directly led to the project's value proposition and engineering requirements.

1.1.5.5 Current (Legacy) Solution and Gap

The current legacy solution for SerDes link training relies on firmware running on an embedded microcontroller, such as Marvell's 8051-based platform, to schedule training tasks, update equalization settings, and monitor link behaviour. This approach has been effective for existing commercial systems because it is compact, familiar, and deterministic, but the

interactions between stakeholders reveal a growing gap. Marvell’s downstream customers require reliable, high-throughput links with minimal downtime, which pushes Marvell toward more complex equalization procedures and stricter firmware timing requirements. At the same time, Professor Sheikholeslami and the academic research community need a public and reproducible platform for studying how link-training algorithms behave under realistic embedded constraints. The capstone team therefore identified a need for a lightweight, deterministic, and portable scheduler that could preserve the low-resource behaviour of the legacy solution while moving toward an open RISC-V platform that is easier to adapt, evaluate, and build upon.

1.2 Value Proposition and General Approach

This project addresses the need for a lightweight, deterministic firmware scheduling platform for SerDes link training. Marvell’s existing link-management firmware operates under strict constraints on memory usage, interrupt latency, and scheduling overhead. As SerDes links become faster and more complex, these constraints become increasingly important because inefficient firmware can increase link-training time, consume valuable on-chip resources, and limit the portability of future training algorithms.

Our proposed value to Marvell is a custom RISC-V-based scheduler designed for resource-constrained SerDes firmware. Rather than replacing the entire link-training system, the project provides a proof-of-concept scheduling architecture that demonstrates how link-management tasks can be executed with low memory overhead, predictable timing, and support for configurable scheduling policies. If developed further, this approach could reduce the area and power cost associated with firmware memory requirements while improving the responsiveness of link-training operations.

The project also provides value to Professor Sheikholeslami and the broader wireline research community by making the link-training problem more reproducible and open to study.

Because commercial SerDes training algorithms and firmware implementations are typically proprietary, there is limited public work connecting adaptive equalization algorithms to the embedded scheduling infrastructure required to run them. To address this gap, our approach combines a simulated SerDes channel, an LMS-based equalization training procedure, and a

schedulable firmware task model. This allows the scheduler to be evaluated in a realistic link-training context without requiring access to a physical SerDes test chip.

The general approach is therefore twofold. First, we model the SerDes link and equalization process in simulation to verify that the training algorithm can converge and improve channel performance. Second, we implement the scheduler in C for a RISC-V target and evaluate its timing and memory behavior using emulation and hardware-oriented testing. Together, these components allow us to assess both sides of the design problem: whether the equalization workload is meaningful for link training, and whether the scheduler can support that workload within the constraints of embedded SerDes firmware.

1.3 Engineering Requirements

As established, a scheduler is required to coordinate the sharing of analog and computational resources while maintaining stable SerDes link training and monitoring. Herein are described the operational constraints under which it must function. Link training consists of a sequence of tightly coupled, time-sensitive operations: adjusting equalization parameters, applying updates to the link, observing receiver response, and determining subsequent actions. In multi-lane systems, these operations must be interleaved across several lanes while also responding to asynchronous events such as resets, data-rate changes, and degraded link conditions. This creates a workload that is both concurrent and interrupt-driven, with strict timing requirements and no tolerance for unbounded delays.

To support this execution model, the scheduler must guarantee bounded latency, deterministic task selection, low memory usage, and minimal implementation overhead. These guarantees are required because the scheduler operates in a resource-constrained embedded environment, tightly coupled to SerDes hardware, where general-purpose operating system abstractions are not feasible. The requirement of low latency is not captured by a single value, but by several distinct timing components (Figure 5):

- **Task creation time:** the time required to add a task to the set of runnable tasks
- **Scheduling decision time:** the time required to select the next task after a task yields
- **Context switch time:** the time required to save the state of the current task and restore that of the next

- **Interrupt latency:** the time from interrupt assertion to the start of the interrupt handler

Scheduling decision time and context switch time occur sequentially, but are treated as separate metrics. Context switch time represents a lower bound imposed by state preservation, whereas scheduling decision time depends on the scheduling policy and can be reduced through simpler selection mechanisms. Treating them independently allows clearer evaluation of trade-offs between scheduler complexity and performance.

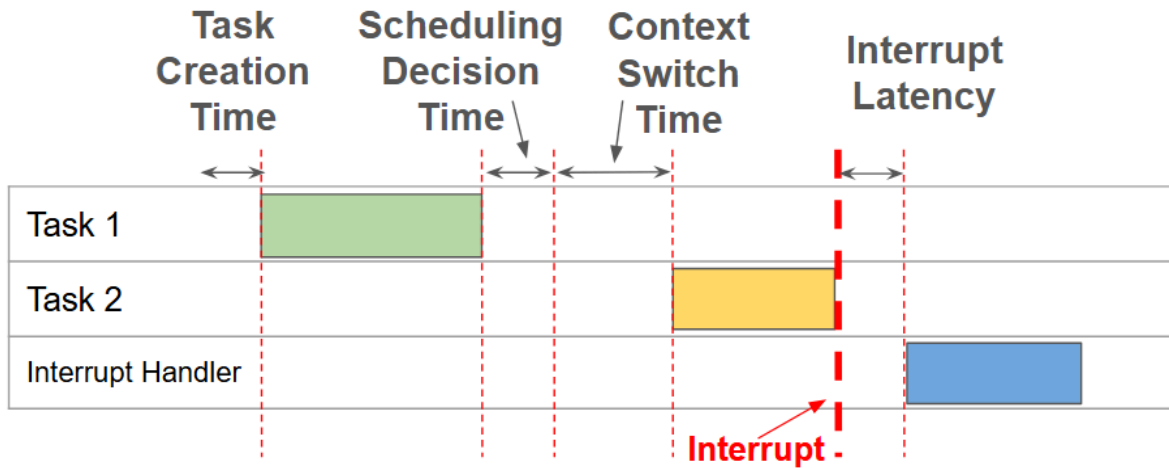


Figure 5: Diagram showing the switching between tasks within a processor, where the time delays of the scheduler are annotated.

These constraints motivate the engineering requirements in Table 1, which evaluate whether the proposed scheduler can support realistic link-training workloads while remaining feasible for embedded SerDes firmware. Based on these functional needs, we develop our own requirements and contextualize the set of requirements provided by Marvell, with a goal to adhere to the guidelines of ISO29148:2018 [X9]: requirements should be necessary, unambiguous, feasible, verifiable, and correct. The numerical targets provided for each metric are provided by Marvell and are based on the performance of Marvell's current proprietary systems, whose specifications are not publicly available.

Table 1: Engineering Requirements			
Objective	Metric	Constraint	Justification
The design should allow for efficient transitions between tasks	Number of CPU cycles or time in micro-seconds	Context switch time is less than 50 CPU cycles	The metrics shown determine the time needed to schedule and begin a new task. If this time is large, the parameters cannot be updated as quickly, leading to greater system downtime. Moreover, a slow time to update might not allow for the capture of changing link conditions due to temperature variations [X10]. The exact numbers are given by Marvell, based on not degrading the performance relative to their current system.
		Interrupt latency less than 20 CPU cycles	
		Scheduling decision time less than 30 CPU cycles.	One important dynamic to be considered is the relationship between the requirements for scheduler performance provided to the design team by Marvell and the initial functional requirements for link performance provided to Marvell by their customers. For example, the latency requirements shown are ultimately informed by functional requirements from Marvell's customers regarding link performance. Marvell's customers approach Marvell with requirements for the amount of downtime for the link, listed as a percentage of total link operation time. Marvell reformulates this requirement into budgeted time constraints for key link reconfiguration operations, including

		Task creation time is less than 100 CPU cycles.	link-retraining. Finally, based on the clock frequency of the microcontroller used by Marvell, these time constraints can be further reformulated into CPU cycle constraints to be used by our design team. Since the design team is not responsible for the choice of microcontroller or clock frequency, only the final CPU cycle constraints need to be considered during the design process. However, a rough estimate for the time constraints can be obtained based on the CPU cycle requirements and the 12 MHz clock frequency used on the 8051 microcontroller, which is the existing 8-bit microcontroller solution used by Marvell. In particular, based on the CPU cycle constraints which are on the order of 10^1 to 10^2 cycles, we estimate that the downtime constraint for individual scheduler operations is approximately 1-10 microseconds.
The design should be memory-efficient	Memory in bytes	Total scheduler memory overhead less than 2 KB Operation within 256 KB of SRAM or DRAM	Marvell and its customers wish to minimize power consumption and area, as both directly impact purchase and operation costs. Therefore, minimizing memory consumption is important throughout a SerDes design, including the scheduler, which is a smaller component. The exact numbers are given by Marvell, based on not degrading the performance relative to their current system.

The design should be cost-effective	Cost (\$)	Do not incur additional licensing costs	Some instruction sets may incur licensing costs while others, particularly RISC-V [X11], do not. The original requirement provided by Marvell was to use RISC-V, but as per ISO29148:2018, we reduce this requirement to its essential quality.
The design should be accurate	Precision	Less than 10^{-16} quantization error	Large quantization errors can lead to errors in LMS training. The original requirement provided by Marvell was to use double-precision floating point, but as per the standard of developing requirements that we use, we resolve the requirement to the fundamental quality of minimizing quantization error. There exist methods of manipulation that reduce the quantization error [X12], [X13], so we set the requirement of achieving equal quantization error as double-precision floating point.

1.3.1 The Need for a RISC-V Scheduler

Marvell’s existing SerDes scheduling approach is based on an 8051 microcontroller, an 8-bit Harvard-architecture platform that has historically provided a compact and deterministic firmware environment. However, as SerDes link training becomes more complex, this legacy platform creates limitations in portability, scalability, and long-term maintainability. A scheduler that is closely tied to a specific microcontroller architecture or toolchain is harder to reuse across future SerDes products, especially as training algorithms, lane counts, and timing requirements evolve.

This creates a gap for a lightweight scheduler that preserves the deterministic behaviour and low resource overhead expected in embedded SerDes firmware (as outlined in table 1), while running on a more flexible and extensible hardware platform. RISC-V is well suited to this role because its open and modular instruction set allows the scheduler to be implemented without dependence on a proprietary processor architecture. Therefore, the design gap addressed by this project was the development of a deterministic, memory-efficient SerDes firmware scheduler capable of running on a RISC-V hardware platform.

1.4 Broader Implications and Ethical and Legal Considerations

When projects have such far-reaching implications for industry, especially one more concealed such as high-speed wireline that underpins rapid communication that is often proprietary, care must be taken to consider the ethical and legal implications and ensure the project appropriately addresses these concerns.

1.4.1 Implications for Industry, Academia, and Society

From an industrial perspective, this project contributes to the development of more scalable and maintainable firmware architectures for high-speed wireline communication systems. SerDes links are used extensively in data centres, networking hardware, and chip-to-chip communication, where increasing data rates place greater demands on link training and calibration. A lightweight and deterministic RISC-V scheduler could help SerDes vendors reduce dependence on legacy microcontrollers and proprietary firmware platforms while improving portability across future embedded systems. In the long term, this type of

scheduler could make link-management firmware easier to modify, verify, and adapt as equalization algorithms evolve.

From an academic perspective, the project addresses the limited public availability of firmware-level SerDes link-training implementations. While equalization techniques are widely studied, the scheduling infrastructure required to run adaptive training algorithms under embedded hardware constraints is less visible in open literature. By developing a reproducible simulation framework and firmware-compatible scheduler prototype, this project provides a foundation for studying the interaction between equalization algorithms, embedded scheduling, and hardware resource constraints. This may support future research in algorithm-scheduler co-design for high-speed communication systems.

From a societal perspective, improvements in wireline communication infrastructure support the continued growth of data centres, cloud computing, artificial intelligence, and high-performance networking. These systems increasingly depend on reliable and efficient high-speed interconnects to move large volumes of data between chips, boards, and servers. Although the scheduler developed in this project is only one component of a much larger communication system, more efficient link-training firmware can contribute to reduced link downtime, improved system reliability, and better use of existing hardware resources.

1.4.2 Environmental Considerations

The environmental impact of this project is indirect but still relevant. Data centres and networking infrastructure consume significant energy, and SerDes links are deployed at very large scale within these systems. A scheduler with lower memory overhead and reduced execution latency may help reduce the area and power cost of the embedded controller used for link management. While the impact of a single scheduler instance is small, improvements at the firmware and hardware level can become meaningful when replicated across many lanes, chips, and systems. At the same time, this project should not overstate its environmental benefit: making high-speed communication more efficient can also support the continued expansion of data-intensive computing infrastructure. The environmental value of this work is therefore best understood as a contribution to efficiency within a broader industry that must still address overall energy use and lifecycle impacts.

1.4.3 Safety, Reliability, and Ethical Considerations

Safety and reliability are important because SerDes links are not limited to consumer or data-centre applications. High-speed wireline communication can also appear in automotive, medical, aerospace, and industrial systems, where communication failures may have more serious consequences. In these contexts, deterministic firmware behavior is important because link training, retraining, and interrupt handling must occur within predictable timing bounds. A scheduler that has bounded latency, low memory usage, and verifiable behavior is therefore preferable to a less predictable software stack. Although this project is a proof-of-concept rather than a production safety-certified design, the emphasis on deterministic scheduling and measurable timing behavior is aligned with the needs of safety-critical embedded systems.

There are also ethical considerations related to transparency and reproducibility. Commercial SerDes firmware is often proprietary, which limits the ability of researchers and smaller organizations to study practical link-training implementations. By keeping the project methodology reproducible and avoiding confidential implementation details, the project supports open academic investigation while respecting the intellectual property boundaries of industry partners. This balance is especially important because the project proceeds in collaboration with Marvell without a non-disclosure agreement. As a result, the team avoids relying on confidential design information and instead frames the work around public methods, general design constraints, and independently developed prototypes.

1.4.4 Legal, Standards, and Institutional Considerations

The deployment of high-speed communication systems is shaped by standards bodies and industry consortia that define interoperability, compliance, and testing requirements. Organizations such as IEEE, OIF, and PCI-SIG influence the electrical specifications, channel models, and validation procedures used in commercial SerDes systems. While this project does not attempt to implement a full standards-compliant SerDes product, future development would need to consider how the scheduler interacts with standardized link-training procedures, compliance testing, and firmware timing requirements. Adoption in an industrial setting would also require further validation, documentation, and integration with the company's internal verification and IP-review processes.

1.5 Project Timeline

January: The team began with background research on SerDes architectures, RISC-V firmware development, embedded scheduling algorithms, and adaptive equalization methods. During this stage, the project scope was refined, stakeholder objectives were clarified, and the main technical constraints were identified.

February: The team developed and validated the initial MATLAB-based SerDes simulation and phased link-training algorithm. This work established the equalization model used to evaluate link-training behavior and helped translate the algorithm into a task-based structure suitable for firmware scheduling. During this period, the team also defined the scheduler architecture, key performance targets, and the interface between the scheduler and the equalization workload.

March: The team implemented the firmware scheduler in C, with RISC-V-specific components used where necessary to support low-level context switching and timing control. The scheduler was cross-compiled for a RISC-V target and functionally validated using the QEMU emulator. In parallel, the team worked on integrating the scheduler with the link-training task model and began preparing the RISC-V hardware implementation pathway.

April: The team focused on system-level integration, performance characterization, and final optimization. Testing was used to evaluate the scheduler's timing behavior, memory usage, and ability to support representative link-training tasks. The final stage of the project involved consolidating the results, preparing documentation, and presenting the completed proof-of-concept design.

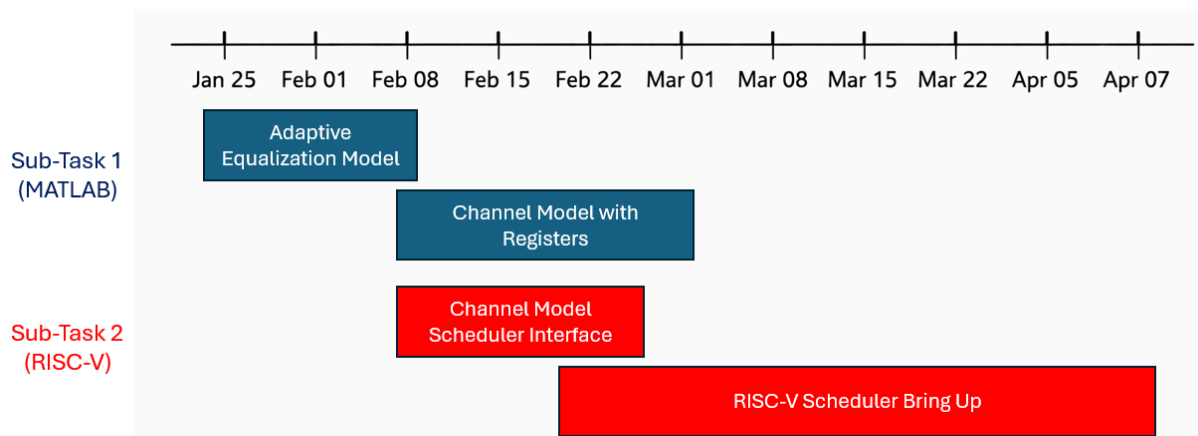


Figure 6. Initial project timeline developed in late January detailing expected progress throughout the project. The group expected to work with only Matlab and RISC-V on the FPGA, and would develop these in two stages.

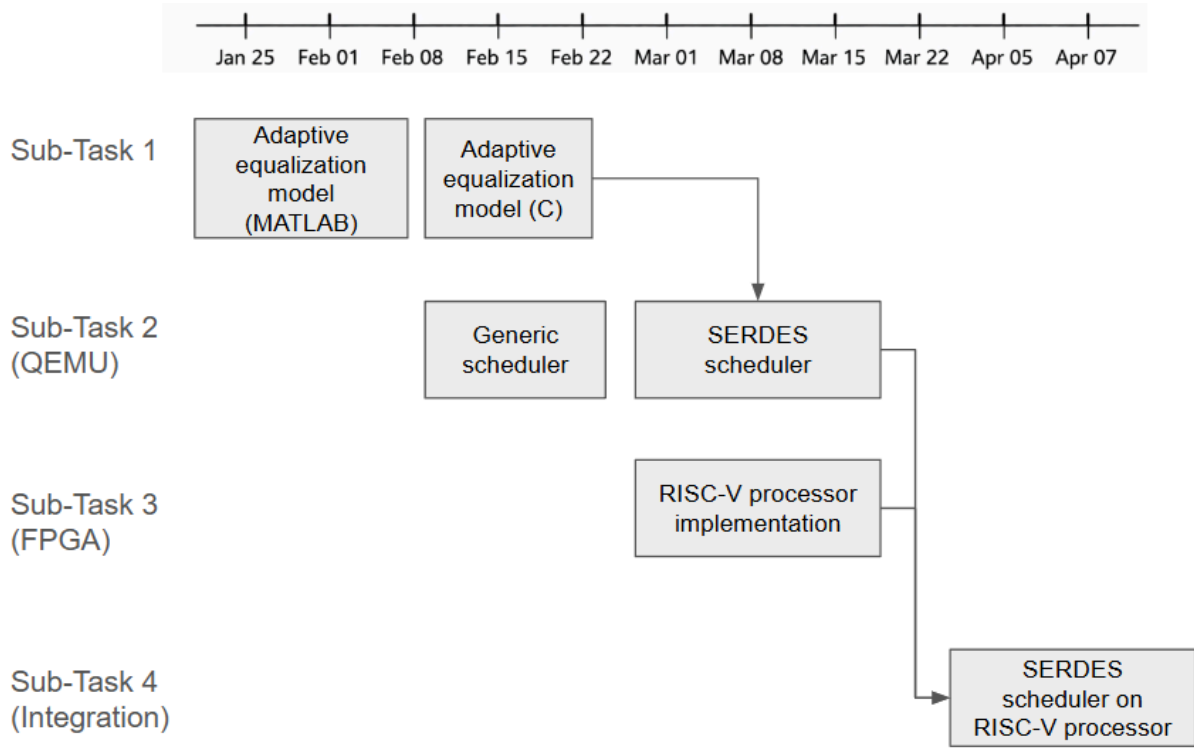


Figure 7. Final project timeline detailing the progress undergone by the team throughout the project. The final project combined the RISC-V processor with the SerDes scheduler, which runs the adaptive equalization model in C.

Figures 6 and 7 summarizes how the project was projected to progress and actually did progress during this project respectively. As the project direction was constantly informed by technical mentors and academic mentors from Marvell and Professor Sheikholeslami's research group, the project evolved

The MATLAB equalization model was developed first because it provided the link-training workload needed to evaluate the scheduler in a realistic SerDes context, and could take reference from earlier work done from Professor Sheikholeslami's group. Afterwards, in order to properly run in the QEMU emulator, the models were translated from Matlab into C, and a generic scheduler was developed concurrently.

Once the channel model and equalization algorithm were established, the team used them to define the task structure that the firmware scheduler would eventually manage. The scheduler

and equalization model were then brought together through an interface that allowed the RISC-V-side implementation to interact with the simulated link-training process. This staged approach allowed the team to validate the algorithmic behavior of the equalization model before moving toward the more constrained RISC-V implementation.

Chapter 2. Conceptual Design

2.1 Scheduler design considerations

In this section we introduce the technical requirements of SerDes which inform our design considerations for the link training. We briefly discuss what a scheduler informed by SerDes equalization would look like when compared with a generic scheduler.

We have established that a gap exists for a low memory scheduler capable of training SerDes links; however, we now consider how the task of training a SerDes link influences the design of our scheduler. To understand how SerDes shapes our design concept, we consider the four different approaches to developing a scheduler. First, we consider existing open source schedulers and discuss whether they can meet our clients' needs; we then consider a customized programmable scheduler, whose design is informed by SerDes but no equalization algorithm specific details are hardcoded into the scheduler; we also consider a scheduler which does hardcode portions of the equalization algorithm for performance boosts; and finally, we explore some non-scheduler alternatives. These options are summarized in Figure 8.

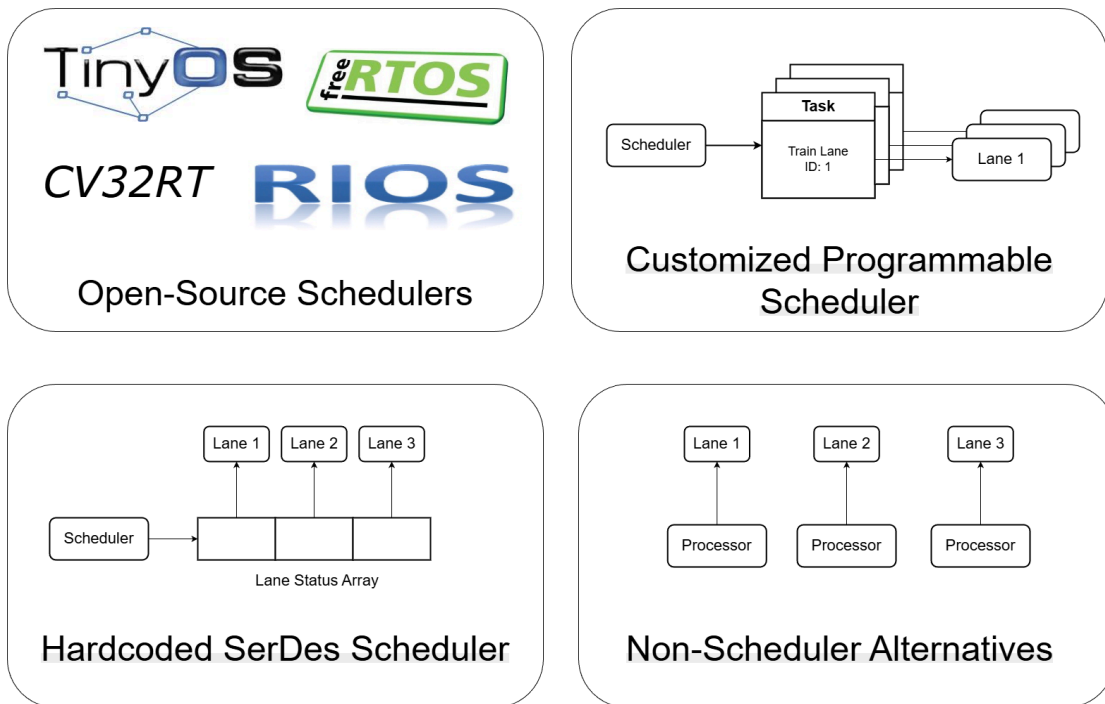


Figure 8. Visual summary of the four potential scheduler designs for this project

2.1.1 Available Open-Source Schedulers

We conduct a poll of existing lightweight hardware schedulers that may be applicable to our context, summarized in Table 2.

Table 2: Available open-source task scheduler solutions and an analysis of each with reference to our engineering requirements.		
Scheduler	Architectural Analysis	Performance Analysis
FreeRTOS	FreeRTOS is a typically-used lightweight operating system for microcontrollers. While there is RISC-V support for FreeRTOS, we do not require the full operating system - we only require a scheduler. While we could separate the scheduler from the operating system in FreeRTOS and repurpose it for SerDes, this scheduler was designed with a larger operating system in mind and hence has unnecessary overhead that would not serve any purpose in our application.	The FreeRTOS scheduler uses roughly 236 bytes and maintains an interrupt latency of 10 to 16 cycles under specific test conditions, meeting our requirements within these metrics. It has a context switch time of 84 cycles, which misses our goal of 50 cycles. FreeRTOS documentation discusses that they will save an estimated 12 CPU cycles of context by switching to assembly, but avoid this in exchange for code simplicity and maintainability [X14]. [X15]. These results are obtained using a high-performance, 32-bit, ARM Cortex-M3, which uses 3-stage pipelining, and an average CPI of one cycle per instruction [X16], suggesting that performance on our hardware-constrained environment would be even worse.
University of California	RIOS is simple and easy to understand, making it a good candidate for early prototyping. However, it adopts an	RIOS memory overhead is roughly 1 kB, so it satisfies our memory overhead requirement [X17]. RIOS is fully

Riverside, RIOS	unconventional design in which tasks are executed directly within the interrupt service routine (ISR). While this approach reduces scheduling complexity, it is only suitable for very short, bounded tasks. For more complex operations, such as SerDes training, executing within the ISR can significantly increase interrupt latency and may lead to violations of real-time timing requirements.	cooperative so there are no context switches or interrupts, which we would require for our design.
TinyOS	Similar to FreeRTOS, TinyOS is an operating system made to handle the resource-constrained environment of sensor networks. TinyOS does not use RISC-V but rather its own language, nesC.	TinyOS is an operating system made to handle the resource-constrained environment of sensor networks. TinyOS contains a scheduler that achieves an interrupt latency of 8 CPU cycles and a memory overhead of less than 400 Bytes, both succeeding in our requirements. The context switch time is 108 CPU cycles, which misses our requirement [X18].
CV32RT	This is a custom hardware core, and the hardware cannot be changed from what Marvell already has, therefore this option is infeasible for our purposes.	Maintains interrupt latency of less than 10 clock cycles, satisfying our requirement but has task creation time of just over 100 clock cycles, which misses our 50-cycle requirement. They create a customized RISC-V core to allow for fast interrupts, that includes customized handling of back-to-back interrupts by skipping unnecessary steps [X19].

2.1.2 Hardcoded SerDes Scheduler

The hardcoded SerDes scheduler operates in a tick-based paradigm. In each tick, the next task is selected based on the configurable scheduling policy, however, instead of dispatching a generic link training task, a specialized link training task, representing one of the several lanes is executed. This involves loading a “lane context” from memory which represents the current, real-time progress of the link training for a given lane, and actually completes the computations involved in the relevant step of the link training process to advance the progress of the lane. Finally, the context switch involves loading and unloading the relevant lane contexts from memory instead of simply invoking a generic C function which represents the next step in the training process. Figure 9 (left) provides an illustration of the hardcoded SerDes scheduler.

2.1.2.1 Hardcoded SerDes Scheduler with Block Coordinate Descent

The optimization scheme of block coordinate descent (BCD) may be leveraged within a hardcoded SerDes scheduler to improve convergence speed. BCD partitions the optimization variables into blocks, iteratively optimizing one block while holding the others constant. A block selection rule can be employed to prioritize the block expected to yield the greatest improvement, for example by selecting the block with the largest gradient [X20]. This approach has previously been applied in domains such as movable antenna optimization, where TX positions, RX positions, TX beamformers, and RX filter coefficients are treated as separate blocks [X21]. By analogy, SerDes equalization parameters—including CTLE, RX FFE, TX FFE, and DFE coefficients—could be partitioned into blocks and optimized sequentially under a similar framework. A scheduler implementing BCD would therefore require a block selection policy to determine which parameter group to update at each step, potentially improving convergence behavior or final link quality.

However, this approach is not pursued further. Its effectiveness depends on the ability to arbitrarily control which subset of equalization parameters is updated at a given time. In practice, this assumption does not hold: the adaptation mechanisms governing these parameters are embedded within Marvell’s proprietary equalization algorithm, whose internal

update rules, coupling between parameters, and stability constraints are not exposed. Imposing an external block-wise update schedule risks violating these constraints, potentially leading to degraded convergence or instability. Without visibility into or control over the underlying adaptation process, a BCD-based scheduling strategy cannot be reliably implemented or validated.

2.1.3 Customized Programmable Scheduler

A customized programmable scheduler is a scheduler made in house that does not encode anything related to equalization explicitly, but whose specification is informed by the use case of equalization. This introduces additional overhead in both memory and execution time from having to load additional jump pointers rather than hard-coding jump targets within the code itself, but provides additional flexibility. For instance, separate dummy tasks can be used to print debugging information to terminal during testing; additional lanes can be easily added by adding additional tasks; different tasks are easier to develop, such as a training task and a monitoring task. Figure 9 (right) illustrates how the customized programmable scheduler would function with different tasks.

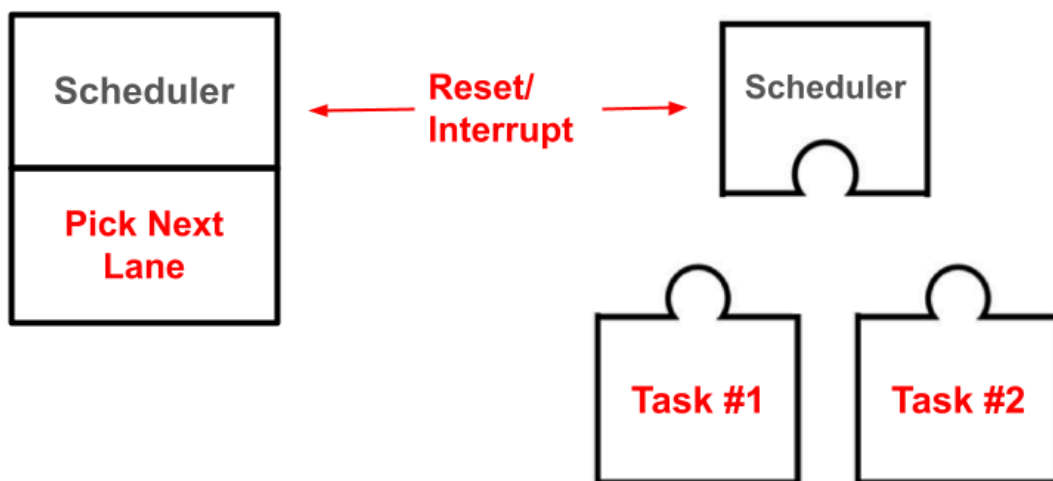


Figure 9: Hardcoded SerDes scheduler (left) vs. customized programmable scheduler (right).

2.1.4 Non-Scheduler Alternatives

Rather than a central scheduler, each lane could be provided with its own processor that runs through an FSM which includes all stages of equalization adaptation. This FSM could be

rerun at specific intervals based on how long we expect the link conditions to change over. Shared resources such as the PLL could have a copy given to each lane so there are no conflicts. Though eliminating the need for a well-designed scheduler, this solution would be prohibitively expensive in terms of cost due to the additional resources required in order to avoid resource sharing, thus violating our low-cost objective.

Closely related to no scheduler, is the reduction of context switching and interrupts by running many parallel hardware threads to handle each task [X22]. Though this may help eliminate some of our requirements, it fails within our context where we are unable to increase the number of hardware threads. Once again, the limited resources available encourage the use of a scheduler to optimize resource sharing.

2.2 Selected Conceptual Design: Customized Programmable Scheduler

Based on the design comparison in Section 2.1, the team selected a customized programmable scheduler as the final conceptual design. This option best satisfies the project's engineering requirements while also addressing the needs of the project stakeholders. The evaluated open-source schedulers provided useful reference points, but each failed at least one essential requirement for this application. Some exceeded the required context-switch latency, while others relied on architectural assumptions, such as running tasks directly inside interrupt service routines or requiring broader operating-system infrastructure, that are not well suited to SerDes link-training firmware. Similarly, eliminating the scheduler would avoid scheduling overhead, but would require additional hardware resources or duplicated lane-control infrastructure, conflicting with the project's low-resource objective.

2.2.1 Justification for Selected Conceptual Design

The customized programmable scheduler satisfies the requirements more directly because it can be designed specifically around the SerDes firmware workload. Its static task structures, bounded task-selection logic, cooperative yield points, and limited saved context reduce unnecessary overhead compared with a general-purpose scheduler. These design choices make it feasible to meet the required timing targets for context switching, interrupt handling, scheduling decisions, and task creation, while keeping memory overhead low. This is especially important as the number of lanes increases. For example, in a ten-lane training

scenario with a 256-symbol training burst, one scheduler activation per lane per symbol would require 2,560 scheduler activations. An added overhead of only 50 CPU cycles per activation would consume 128,000 additional cycles, which corresponds to approximately 10.7 ms on a 12 MHz controller. This estimate shows that the cycle requirements are not arbitrary; meeting them is necessary for the scheduler to remain practical as link-training work scales across lanes and repeated update steps.

A hardcoded SerDes scheduler could also be designed to meet the timing and memory requirements for one known training procedure. However, it would do so by embedding assumptions about the equalization algorithm, training order, lane-management policy, and contents of each training step. These details may change as Marvell modifies its link-training methods or develops future SerDes products. The customized programmable scheduler therefore provides a stronger overall fit: it meets the timing and memory constraints expected of SerDes firmware, while still allowing users to define tasks and yield points for different training algorithms. In cases where a fixed training sequence is desired, the user can configure the programmable scheduler to behave similarly to a hardcoded scheduler, but without permanently sacrificing flexibility. For these reasons, the customized programmable scheduler was selected as the conceptual design.

2.2.2 Interactions with Larger Operational Environment

The entity relationship diagram shown in Figure 10 illustrates the interactions between the firmware scheduler and the operational environment.

The link supports 2 modes of operation: (1) regular link operation which entails data transfer between chips for application-specific purposes and (2) firmware scheduler operation during link training upon startup or link retraining. A link arbitration program runs perpetually, on-chip and is responsible for managing access to the SerDes link between the regular application and the firmware scheduler.

During regular application operation, real-time link conditions are reported asynchronously to the arbitration program so that link re-training can be triggered when link performance deteriorates below a specified threshold.

During startup link training and retraining, the arbitration program transfers access to the link from the application to the firmware scheduler. A queue of tasks and interrupts is generated based on the pending set of operations and the scheduler is left to address them in optimal order by sending test sequences across the channel and measuring error based on receiver output. The scheduler eventually stops running and yields control of the link back to the arbitration program once all links have been trained and all pending interrupts have been addressed.

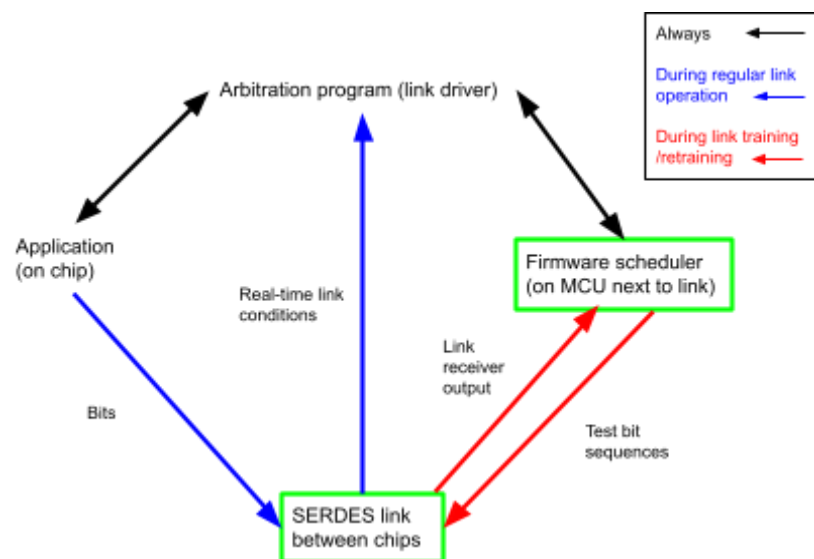


Figure 10: Entity relationship diagram detailing the interactions between components of the SerDes link and their modes of operation. The component designed in this capstone project is the firmware scheduler and a simulation of the SerDes link between chips

2.2.3 Potential Behaviors

Scheduling Policies - While the scheduling policies are left configurable as a project constraint imposed by the Marvell team, we control which scheduling policies are included in our scheduler (in addition to round robin, pre-emptive and hybrid scheduling). We describe the scheduling policies that we consider for our scheduler in this section.

Table 3: Description of Scheduling Policies	
Scheduling Policy	Description of Policy Behavior

Round Robin	Tasks are executed in a fixed cyclic order. Each task gets a turn to run, usually for a limited time slice or until it yields, after which the scheduler moves to the next task in the queue. This is simple and fair, but not always optimal for urgent or time-critical tasks.
Pre-emptive	The scheduler can interrupt a currently running task and switch to another one, usually based on a timer tick or task priority. This allows fast response to high-priority events, but requires more complex context switching and synchronization.
Hybrid	Combines features of multiple scheduling approaches. For example, a system might use round robin among tasks of equal priority, while still allowing pre-emption for urgent tasks. This tries to balance fairness, responsiveness, and implementation cost.
Bitmap	Tasks are associated with bits in a bitmap, where each bit indicates whether a task is ready to run. The scheduler quickly checks the bitmap to find the highest-priority ready task or the next active task. This is efficient and low-overhead.
Co-routine	Tasks run cooperatively rather than being forcibly interrupted. A task continues running until it voluntarily yields control, at which point another co-routine may run. This greatly simplifies scheduling and context switching, but one badly behaved task can block the whole system.

The bitmap and co-routine scheduling algorithms would be beneficial to include in a SerDes context because tasks are all designed by the same user, who would have an incentive to design tasks which interact cooperatively rather than hogging processor time to the detriment of other tasks. Scheduling policies can therefore trust the individual tasks to yield reliably rather than forcing tasks to give up the processor as in round robin. Nonetheless, we do include round robin and other fair scheduling routines due to their simplicity and effectiveness.

Interrupt Handling - We are building a tick-based scheduler, which means the scheduler decides on its course of action at specified timing intervals called ticks (as described in 2.2.2).

When an interrupt occurs, the interrupt is added to a service stack, which is checked at every tick. The scheduler then draws the next task from the service stack whenever the stack is non-empty.

Modes of Failure

Table 4: Possible modes of failure for the custom programmable scheduler	
Failure Mode	Description
Task Starvation	Task starvation occurs when a task is left unscheduled for too long because of an error or inefficiency in our scheduling policy and next-task selection algorithm. Even if we develop our system optimally, starvation can still occur in a wireline SerDes context due to length of training required for certain lanes which may be worse off than other lanes in terms of noise and loss.
Deadline Miss	A deadline miss occurs when a task completes later than its required time. This can happen if the scheduler overhead is too high or if too many interrupts are received by the scheduler, preventing regular tasks from executing as normal, also known as an interrupt storm.
Deadlock or Livelock	This happens if more than one task is left waiting for resources held by another or if tasks remain active but make no progress because they repeatedly react to each other.
Priority Inversion	This occurs if a low-priority task blocks a high-priority task because it holds a shared resource.
Thrashing	Thrashing can occur if the scheduler spends more time performing context switches than executing link training. This would cause the system to become bottlenecked by the speed of memory accesses, a phenomenon commonly referred to as the von Neumann bottleneck.
Priority Collapse	When too many tasks share the same priority, effective scheduling degenerates, because the scheduler resorts to FIFO ordering of tasks

	without regard for their relative importance.
--	---

2.2.4 User Interactions

Application Programming Interface (API) - Our decision is to make a generic scheduler optimized for context constraints, not one hardcoded for link scheduling. Therefore, the user must be given commands that they can use to interface with the scheduler. A tentative list of commands is given in appendix B, which are adapted from FreeRTOS [X23].

Application Binary Interface (ABI) - In order to meet the engineering design requirement of using only 50 cycles for a context switch, we must limit the number of registers that we save when saving our context. In 32 bit RISC-V, there are 32 registers to save in addition to CSR registers which describe program state, and each register requires on average more than 1 clock cycle to save. Therefore, saving all registers would result in over 50 clock cycles per context switch. In order to meet our engineering requirements, we only save 16 registers, and leave the remaining 16 to be overwritten. Hence, we impose a calling constraint in the form of an ABI, where functions can only assume the first 16 registers will be restored following a context switch, and users must adapt their code around this constraint.

Interrupt Service Routine (ISR) Interface - Similarly, we cannot save every register during an interrupt service request - due to our constraint of handling these requests with only 20 cycles of latency, we cannot even save 16 like in the case of a context switch. Therefore, we save only 4-8 registers that may be overwritten during an interrupt handling. It is then a requirement of the user to only work with the saved number of registers when handling interrupts. Again, this is a constraint imposed on the user by our design choice.

2.3 Proof-of-Concept Considerations

In this section, we explain our approach to developing and evaluating a custom programmable scheduler that runs on a RISC-V processor. We structure our prototype development into two phases - first, we focus on simulation to evaluate correctness using the QEMU hardware emulator; then, we implement our scheduler on an FPGA RISC-V processor to evaluate performance by counting clock cycles.

Simulation Steps - In order to support rapid iteration in our design process, we require a platform where we can compile and test code quickly for correctness. It would be prohibitively slow to upload our scheduler code repeatedly to a RISC-V board, and it would be prohibitively expensive to buy a RISC-V machine to prepare code and compile on directly. Therefore, we use emulation - QEMU is capable of compiling and running C code in a simulated RISC-V environment. Additionally, we are able to implement verbose logging and see the output directly in our terminal, simplifying the debugging process and allowing us to test for correctness.

Rationale for FPGA Implementation - An FPGA platform was selected instead of an off-the-shelf microcontroller because the goal of the prototype is not only to run scheduler code, but to evaluate how the scheduler behaves on a configurable RISC-V processor architecture. A fixed microcontroller would constrain the evaluation to one specific processor, memory system, interrupt controller, and peripheral set, making it difficult to separate scheduler behaviour from implementation-specific hardware details. In contrast, an FPGA allows the team to synthesize an open-source RISC-V soft core, control the processor configuration, modify the memory and peripheral environment, and evaluate cycle counts in a bare-metal setting. This makes the FPGA a better intermediate platform between QEMU emulation and a future custom embedded SerDes controller. It also supports the project's portability objective, since different RISC-V cores can be tested on the same development board without requiring new physical microcontrollers.

FPGA Implementation - Once the scheduler is validated for accuracy in the emulator, the team will use an FPGA development board to evaluate the performance of the firmware scheduler implementation. An open-source RISC-V soft-core processor will be selected, synthesized, and programmed onto the board to enable precise measurement of cycle counts for critical operations and memory usage, ensuring compliance with engineering requirements. The team currently has access to an Altera DE10-Lite FPGA board, which uses a MAX 10 FPGA.

Three candidate cores represent different trade-offs between area and execution fidelity.

SERV is an extremely area-optimized, bit-serial RISC-V implementation, reported at

approximately 239 LUTs / 164 FFs on Intel Cyclone 10LP in a minimal configuration, making it the most feasible option for tight resource constraints but with significantly reduced performance. **PicoRV32** offers a middle ground, with a compact yet higher-performance design (roughly 750–2000 LUTs on Xilinx 7-series), and can be further reduced through configuration options such as disabling upper registers, counters, and using a single-port register file. **NEORV32**, in contrast, provides a more feature-complete microcontroller-style system with configurable components such as caches, multipliers, and peripherals. While it incurs higher resource usage than SERV and PicoRV32, it enables a more realistic evaluation of scheduler behavior, including interrupt handling and memory-mapped I/O.

Together, these cores allow the team to explore a spectrum of design points: SERV for minimum area, PicoRV32 for balanced efficiency, and NEORV32 for higher system fidelity. The final selection will depend on the desired balance between resource utilization and the accuracy of performance characterization.

Chapter 3. Prototyping and Evaluation

3.1 System Architecture

Production Architecture (Figure X: On-Chip Scheduler and Closed-Loop Adaptation)

Figure X illustrates the system architecture during firmware scheduler operation in a production deployment. The firmware scheduler executes on an on-chip microcontroller and coordinates multiple concurrent instances of the link-training algorithm, each corresponding to an independent SerDes lane.

For each lane, the scheduler manages the iterative update of equalization parameters applied to the receiver front-end. These updates are driven by feedback derived from the received signal, which is processed on-chip to extract error metrics and signal quality indicators. This feedback is routed directly back to the scheduler, enabling closed-loop optimization of equalizer settings.

This architecture enables fully autonomous link adaptation. All training, evaluation, and parameter updates occur locally on-chip, eliminating reliance on external computation. As a result, the system can continuously adapt to channel variations in real time while maintaining deterministic execution and low-latency response.

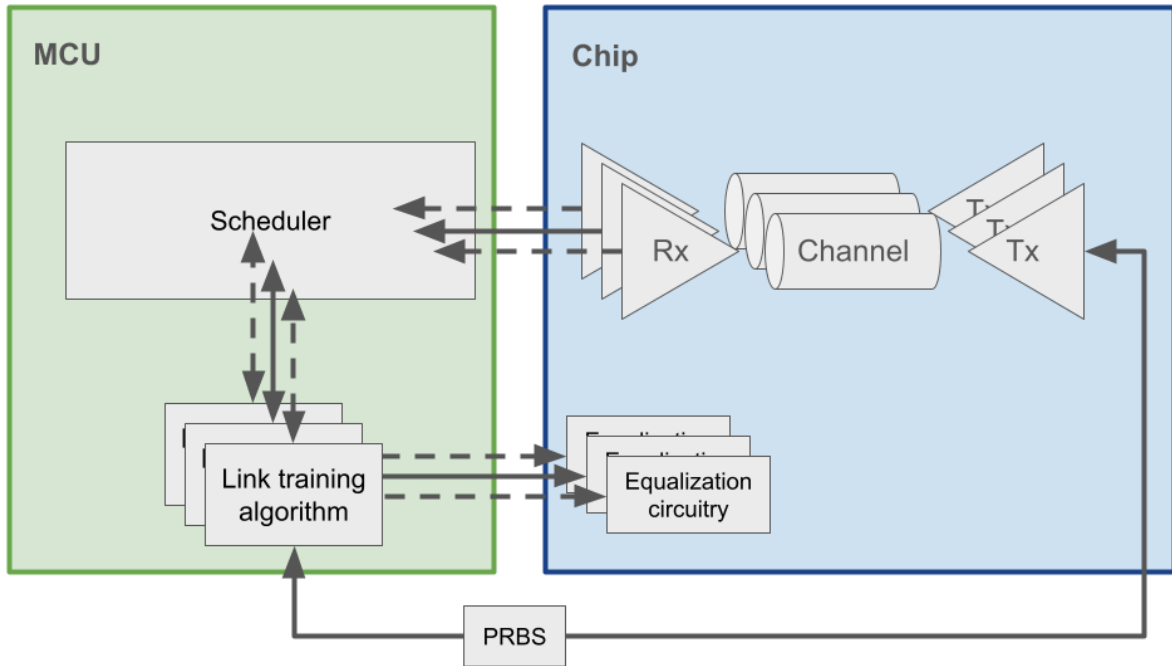


Figure X.

Prototype Partitioning (Figure X: FPGA-PC Co-Simulation Framework)

Figure X shows the partitioning used in the project's prototype implementation. In this setup, the control plane, comprising the firmware scheduler and link-training tasks running on a soft-core RISC-V processor, is implemented on an FPGA, while the channel and equalization models are executed in a PC-based simulation environment.

This partitioning enables rapid development and validation without requiring full hardware integration. The FPGA executes the scheduler in a realistic embedded context, while the PC provides a flexible and observable environment for modeling channel behavior and equalization performance.

Communication between the FPGA and the simulation environment enables real-time exchange of parameters and feedback signals. The scheduler issues parameter updates to the simulated link, and the simulation returns corresponding performance metrics. This preserves the closed-loop structure of the production system while enabling efficient debugging and experimentation.

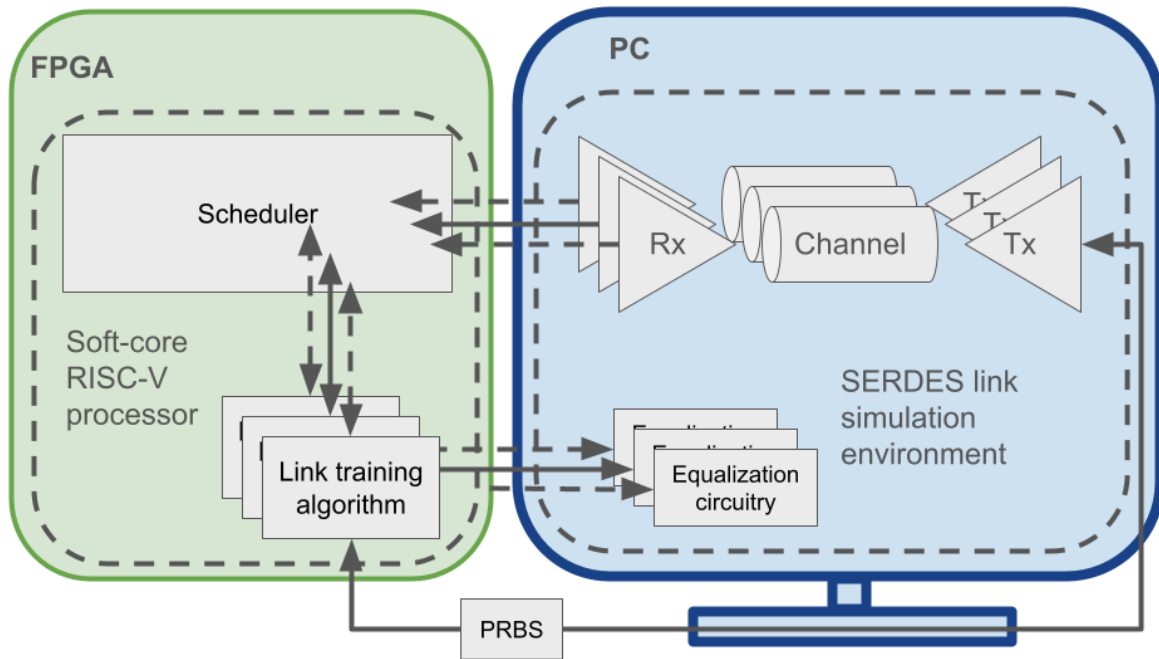


Figure X.

Functional Dataflow (Figure X: Task-Level Link Training Loop)

Figure X illustrates the functional flow of the adaptive link-training process at the task level. At each scheduling step, the scheduler selects a task associated with a specific lane and training stage. The task updates equalization parameters, which are then applied to the transmitter and/or receiver.

A pseudo-random bit sequence (PRBS) is transmitted through the channel, producing a distorted received signal that is subsequently sampled and quantized. The resulting symbol decisions are compared against the known transmitted sequence to compute an error metric.

This error is returned to the scheduler as task feedback, guiding subsequent parameter updates. The process repeats iteratively across lanes and training stages until convergence criteria are met.

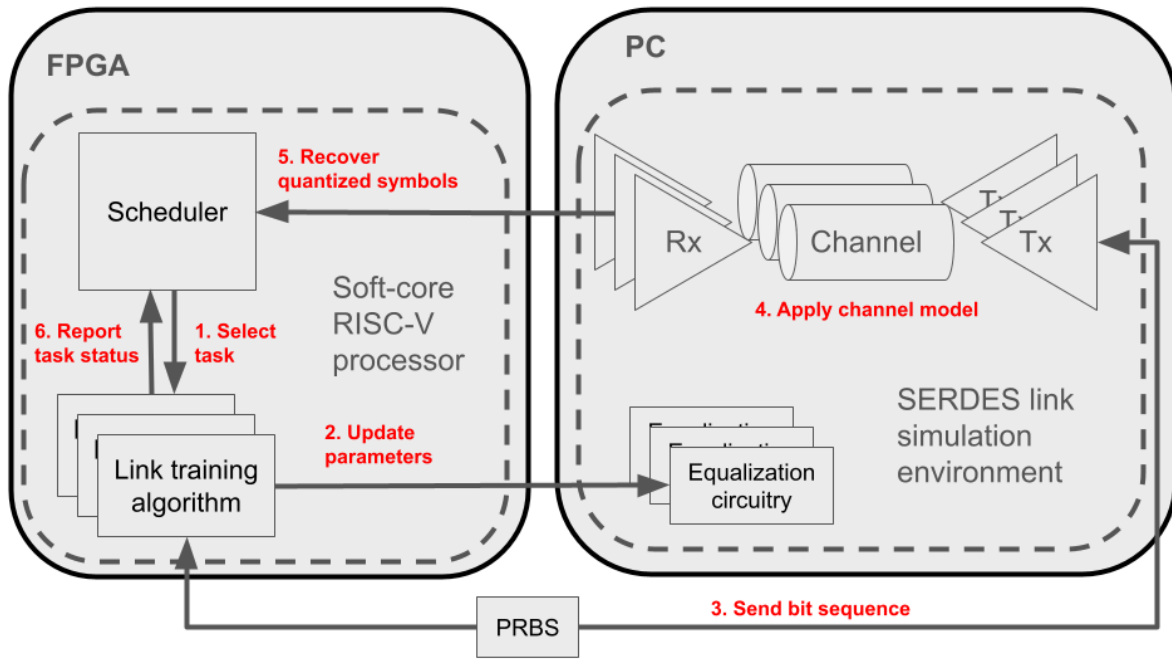


Figure X.

3.2 Implementation Summary

The link training algorithm can be formulated as a sequence of firmware-level tasks, such as measuring receiver error metrics or updating a specific equalization block. These tasks are dynamically generated by the optimization algorithm and submitted to the scheduler, which is responsible for ordering execution, managing context switches, handling interrupts, and enforcing timing constraints.

The general development approach follows a three-phase methodology:

1. Link and transceiver behavior are simulated in MATLAB using a SerDes testbench, where a firmware-feasible link training algorithm is developed and validated.
2. The validated algorithm is migrated to C and integrated with the custom firmware scheduler in QEMU.
3. The combined scheduler and algorithm are deployed on a RISC-V soft core implemented on an FPGA, enabling evaluation of real execution behavior, timing determinism, and resource usage.

3.2.1 MATLAB SerDes Testbench

A MATLAB-based SerDes testbench was developed to validate the equalization workload before deploying it on the RISC-V scheduler. Rather than modelling the scheduler first, the MATLAB testbench provided a controllable software environment for studying the link-training algorithm itself. The script generated a known training sequence, oversampled it, passed it through a fitted channel impulse response, and recovered an appropriate sampling phase using a simple clock-data-recovery procedure. This created a repeatable simulated link on which different equalization stages could be trained and evaluated.

The training process was organized to reflect the staged structure of a practical SerDes link. First, the channel response was imported from an S-parameter channel model and converted into a finite impulse response representation. An optional transmitter FFE stage could then be trained using LMS updates, although the main proof-of-concept focused on receiver-side adaptation. The CTLE parameters were selected by sweeping candidate gain and zero-frequency settings and choosing the combination that minimized the measured symbol error over a training window. Once the CTLE setting was fixed, the receiver FFE and DFE were trained using LMS. At each valid symbol decision, the receiver compared the equalized output against the known training symbol, computed an error signal, and updated the corresponding tap weights. The optimized parameters from earlier stages were carried forward into later stages, so the receiver-side training operated on the already-selected channel, sampling, and CTLE configuration.

The MATLAB testbench was useful because it showed what the firmware would eventually need to do on hardware. For each training step, the model generated received samples, compared them against the known PRBS symbols, computed an error value, and used that error to update equalizer parameters. These same operations later became the basic workload that the RISC-V scheduler had to run repeatedly across multiple lanes. In other words, the MATLAB script did not only test whether the equalization algorithm worked; it also helped define the sequence of tasks that the firmware scheduler needed to support. We provide pseudo-code below to outline the MATLAB simulation:

`Initialize simulation parameters and convergence criteria`

```
Generate known PRBS training sequence
Oversample PRBS sequence so to make channel applicable
Load channel model and transmit PRBS through channel
Estimate receiver sampling phase using CDR

Sweep CTLE settings:
    Apply CTLE
    Sample received waveform
    Compute error against known PRBS
    Select CTLE setting with minimum error

Initialize Rx FFE and DFE taps
while (error > threshold and iteration < max_iterations):
    Sample and quantize received signal
    Apply Rx FFE and DFE
    Compute error against expected PRBS symbol
    Update Rx FFE and DFE taps using LMS
    Record convergence error

Return optimized equalizer parameters and final link-quality
metrics
```

We show some metrics output by our MATLAB simulation below. First, the CTLE training error is provided. Recall that the CTLE is optimized using a grid search/sweep, as one cannot easily obtain the analytical derivative of the CTLE training error. The CTLE is applied multiple times, each with different parameters, and the set of parameters that give the least error over its operation is chosen. The variance in error across the run is shown in Figure X.

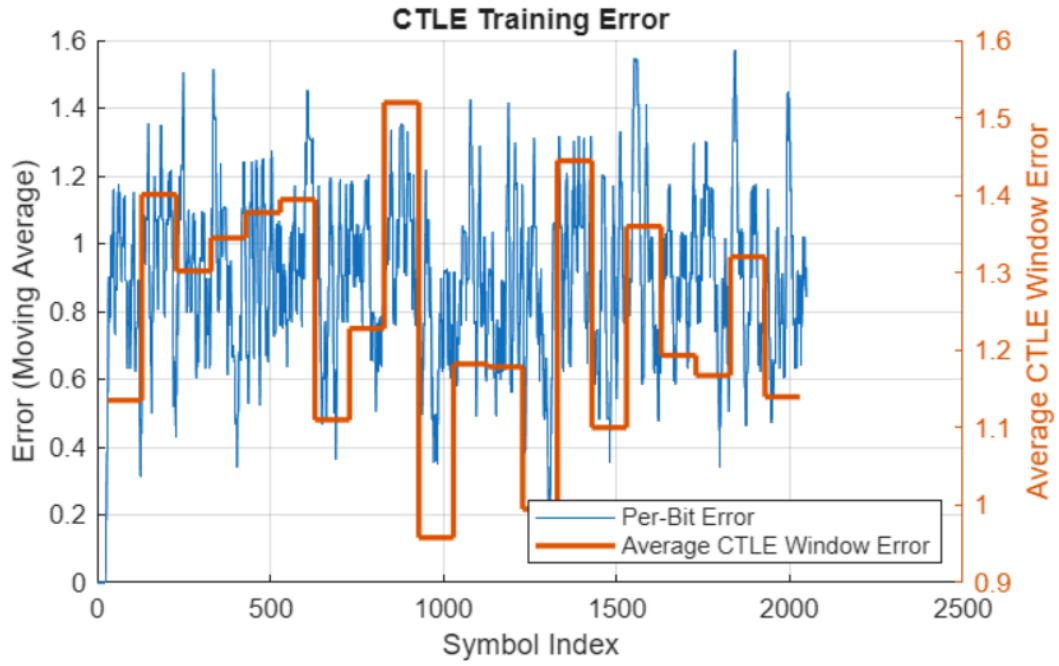


Figure X: CTLE error swept across parameters, with the blue line showing the per-bit error and the orange line the average error for the chosen parameters.

We also show the eye diagram for PAM 2 using the MATLAB simulation for a given s4p file [cite] in Figure X after training. The eye is completely closed at the beginning of the simulation.

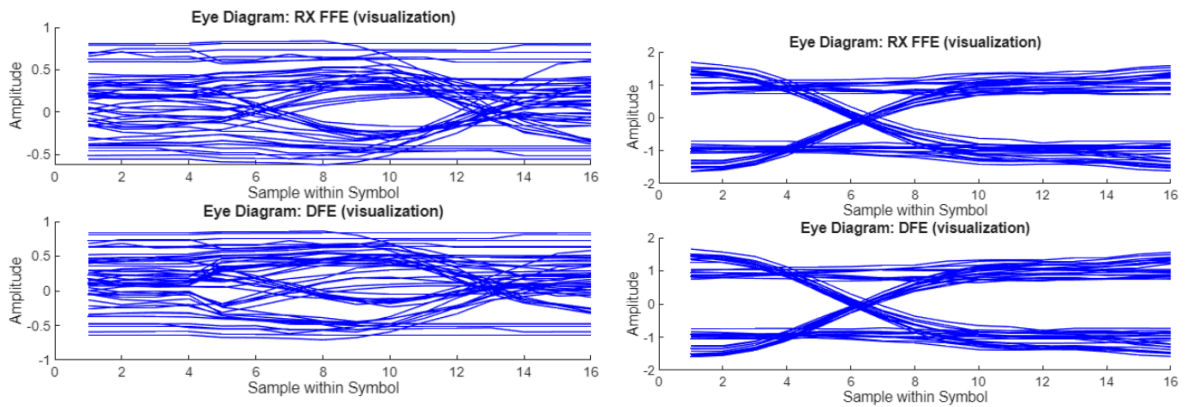


Figure X:

3.2.1.1 Troubleshooting MATLAB Implementation

We attempted to apply the channel response in the frequency domain by multiplying the signal FFT with the channel frequency response, and then taking the inverse FFT. For two reasons, this was problematic; firstly, the FFT and the IFFT produced artifacts that led to an unrealistic output signal. The reason for these artifacts is not well-understood by the team. The second issue was that we wanted to look at the live training/scheduling using the eye diagram, therefore it was better for us to apply the channel pulse-by-pulse. Therefore, we applied our channel in the time-domain by taking the IFFT of the channel model and convolving our signal with this.

3.2.1.2 Removing TX FFE

We do not incorporate FFE at the transmitter in our simulation, as this would imply a backchannel exists over which we can receive information at the transmitter about our signal at the receiver to update our taps. Such a backchannel would not exist in most practical SerDes applications [cite]. Having such a backchannel presupposes the existence of a fully-trained channel.

3.2.2 QEMU Scheduler

We port the MATLAB transceiver model into C so that it can interface with our C-based scheduler. Herein, we describe the scheduler which we validate in QEMU. We provide pseudo-code for the QEMU scheduler below:

```
Timer interrupt handler:
    Save interrupt context
    Check interrupt cause

    if interrupt is not a timer interrupt:
        Restore interrupt context
        Return from interrupt
```

```
Reset timer deadline

if UART communication is currently active:
    Restore interrupt context
    Return from interrupt

previous_task = current_task
next_task = (current_task + 1) mod num_tasks

if next_task is different from previous_task:
    current_task = next_task
    Save previous task context
    Store previous task stack pointer
    Load next task stack pointer
    Restore next task context

Restore interrupt context
Return from interrupt
```

3.2.3 RISC-V Softcore Implementation

The third phase of the prototype development involved deploying the scheduler onto a physical RISC-V processor implemented as a softcore on an FPGA. While the QEMU emulator provided a convenient environment for validating scheduler correctness, it does not model cycle-accurate execution timing or real memory access latencies. To evaluate whether the scheduler meets the engineering requirements for context switch time, interrupt latency, scheduling decision time, and memory overhead, the team required a hardware platform where CPU cycle counts could be precisely measured.

3.2.3.1 Softcore Processor Selection

As discussed in Section 2.3, the team initially considered two candidate softcore processors: SERV, described as the world's smallest RISC-V CPU, and PicoRV32 in a lean configuration.

However, neither processor proved suitable for the DE10-Lite's MAX 10 FPGA. SERV, while extremely compact, implements the RV32I instruction set using a bit-serial datapath that executes one bit per cycle, resulting in a CPI far too high for realistic scheduler timing evaluation. PicoRV32, conversely, was too large to fit onto the MAX 10 device alongside the memory and peripheral infrastructure required by the project; even in its size-optimized configuration, PicoRV32 consumes roughly 750–2000 LUTs on comparable FPGA families, which when combined with the instruction memory, data memory, SDRAM controller, and bus interconnect, exceeded the available logic resources on the DE10-Lite.

The team therefore selected the NEORV32 processor, an open-source, well-documented RISC-V softcore written in VHDL. NEORV32 was the only viable candidate for the DE10-Lite board because it fits within the MAX 10's resource constraints while still providing a complete system-on-chip (SoC) framework. This framework includes a built-in bootloader, UART interface, machine timer (CLINT), on-chip debugger (OCD), and an external bus interface (XBUS), all of which were necessary for the prototype workflow. The bootloader, in particular, allowed the team to upload and test new firmware builds over UART without resynthesizing the FPGA design for each iteration, significantly improving development speed. Table X summarizes the key considerations involved in the choice of the softcore processor.

Table X: Summary of candidate RISC-V processor softcore implementations					
Core	Design Style	Area (Approx.)	Performance Characteristics	Key Features	Suitability
SERV	Bit-serial RV32I	~239 LUTs / 164 FFs	Extremely high CPI (bit-per-cycle)	Minimal implementation, ultra-low area	Not suitable – too slow for timing evaluation
PicoRV32	32-bit RV32I	~750–2000 LUTs	Moderate CPI, configurable performance	Configurable registers, optional features	Not suitable – exceeds FPGA

					resource limits
NEORV32	Microcontroller SoC	Moderate (fits MAX 10)	Lower CPI, more realistic execution	UART, timer (CLINT), bootloader, bus interface	Suitable – best balance of realism and fit

3.2.3.2 FPGA Platform and Clocking

The target hardware was the Altera DE10-Lite development board, which features an Intel MAX 10 FPGA. The board's on-chip 50 MHz oscillator was used as the input clock to a PLL, which generated a 90 MHz system clock for the NEORV32 processor. Reset logic was implemented using a synchronized deassertion scheme: an asynchronous reset, triggered by either of the board's two push-button keys or loss of PLL lock, was passed through a three-stage synchronizer on the rising edge of the system clock to prevent metastability during reset release.

3.2.3.3 Processor Configuration

The NEORV32 was configured with the RV32IMC instruction set: the base integer ISA (I), the multiply/divide extension (M), and the compressed instruction extension (C). The M extension was required because the equalization workload involves multiply-accumulate operations in the LMS tap update, and software emulation of multiplication would have added unacceptable overhead. The C extension reduces code size by encoding frequently used instructions in 16-bit format rather than 32-bit, which is important given the limited on-chip instruction memory. The Zicntr extension was also enabled to provide access to the machine cycle counter (mcycle) and instruction-retired counter (minstret), which were used directly in the scheduler's performance instrumentation to measure cycle counts for context switches, scheduling decisions, and interrupt handling.

3.2.3.4 Memory Architecture

The processor was configured with 64 KB of internal instruction memory (IMEM) and 32 KB of internal data memory (DMEM), both implemented using the FPGA's on-chip block

RAM. These sizes were chosen to accommodate the scheduler code, the equalization task code, and the per-lane state required for multi-lane link training, while remaining within the block RAM resources available on the MAX 10 device. In addition to the internal memories, the design connected the NEORV32's external bus interface (XBUS) to the DE10-Lite's on-board SDRAM through a Wishbone-to-Avalon bridge instantiated using Intel's Platform Designer (Qsys). This provided additional memory capacity that could be used for larger data buffers or expanded lane contexts if needed, although the primary scheduler and task execution paths were designed to operate entirely within the faster on-chip SRAM to avoid the variable latency associated with SDRAM accesses.

3.2.3.5 Timer, Peripherals, and Debugging

The NEORV32's core local interruptor (CLINT) module was enabled to provide the machine timer interrupt used by the tick-based scheduler. The CLINT includes a 64-bit machine timer (mtime) and a timer compare register (mtimecmp), which together generate a periodic timer interrupt at a configurable interval. This mechanism directly implements the tick-based scheduling model described in Section 2.2.3: at each timer interrupt, the interrupt handler saves the current context, invokes the scheduling decision logic, and restores the context of the next selected task. The UART0 peripheral was enabled both for bootloader communication and for runtime output of scheduler diagnostics, including per-tick logging, task status, and convergence metrics from the equalization workload. For debugging and development, the on-chip debugger (OCD) was enabled and connected to the FPGA's JTAG pins. This allowed the team to halt the processor, inspect register and memory state, and set breakpoints during scheduler bring-up, which was particularly useful when validating the correctness of the assembly-level context switch routine on real hardware as opposed to the emulated environment.

3.2.3.6 Firmware Deployment and Portability

The firmware was cross-compiled for the RV32IMC target using the RISC-V GCC toolchain and uploaded to the NEORV32 via its UART bootloader. The same scheduler C code validated in QEMU was recompiled for the FPGA target with minimal modification; the primary changes involved replacing QEMU-specific timer configuration with the NEORV32 CLINT register interface and adapting the UART output routines to use the NEORV32

UART0 hardware registers. This portability between the emulated and hardware environments validated the design decision to write the scheduler in portable C with only a thin layer of RISC-V-specific assembly for context switching and interrupt entry, as described in Section 2.2.4.

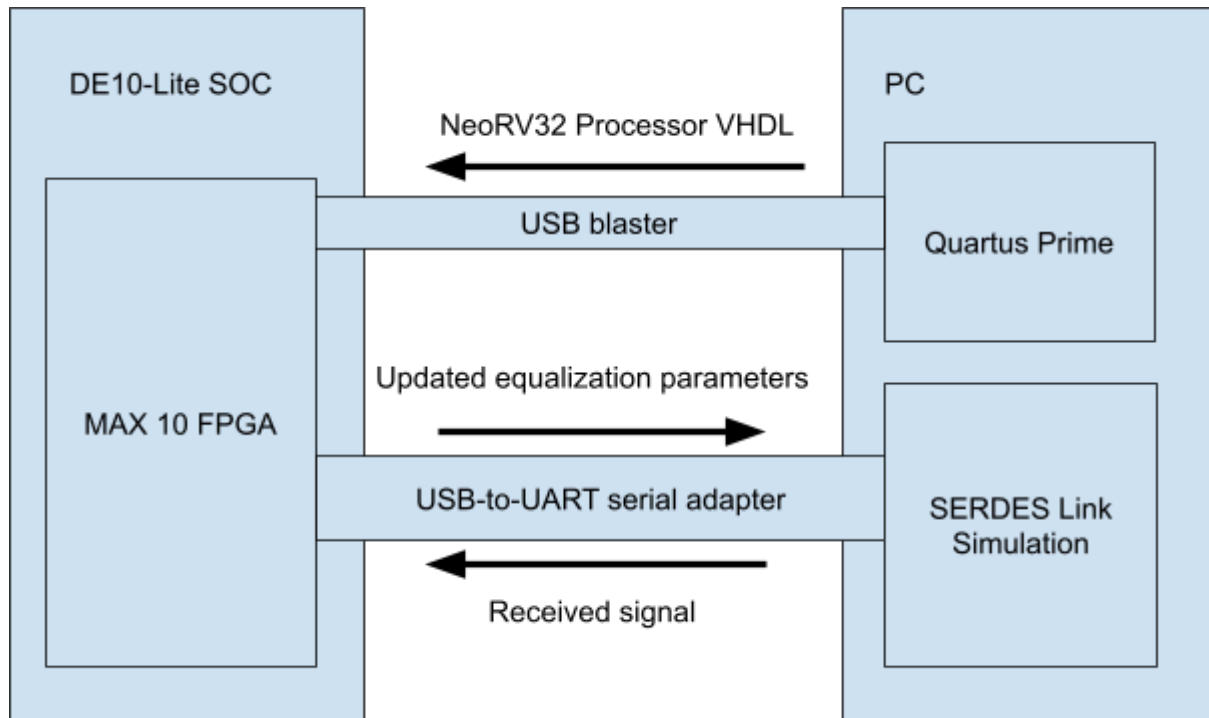
Once deployed on the FPGA, the team used the mcycle counter to instrument critical scheduler operations and measure their cycle counts under real execution conditions. These measurements provided the basis for evaluating the scheduler against the engineering requirements in Table 1 and are reported in Section 3.3.

3.2.4 PC-FPGA Communication Interface

A CP2102 USB-to-UART bridge enables bidirectional communication between the FPGA, which runs the firmware scheduler and link training algorithm, and the SerDes link simulation on the PC. The FPGA transmits updated equalization parameters to the PC, which applies them within the link simulation and returns the resulting quantized received signal. This closed-loop exchange allows the FPGA to compute error and iteratively refine the equalization parameters.

Although this interface is necessary for prototyping, the UART link introduces a significant latency bottleneck. Data transfer between the FPGA and PC is limited by the relatively low baud rate of the UART interface, which increases round-trip time for each training iteration and slows overall convergence of the link training process.

Figure X illustrates the flow of information between the PC and the FPGA in our final prototype as well as the communication protocols and hardware used.



3.2.5 Scheduler Software Performance Optimization

Achieving our requirement of fast transitions between tasks required optimizations to be made to the initial scheduler code, which are herein described.

1. The context switch is implemented directly in RISC-V assembly to avoid compiler-generated function overhead. The routine executes 31 instructions, where the cost is dominated by the 13 register stores used to save the outgoing task context and the 13 register loads used to restore the incoming task context. Since each saved register must be explicitly written to memory and later reloaded, the routine is close to minimal for a software context switch that preserves the callee-saved registers `ra` and `s0-s11`.

```

# Allocate space on the current task's stack for 13 saved
registers
addi sp, sp, -52
# Save the current task's callee-saved registers
sw    ra,    0(sp)

```

```

sw    s0,    4(sp)
sw    s1,    8(sp)
sw    s2,   12(sp)
sw    s3,   16(sp)
sw    s4,   20(sp)
sw    s5,   24(sp)
sw    s6,   28(sp)
sw    s7,   32(sp)
sw    s8,   36(sp)
sw    s9,   40(sp)
sw    s10,  44(sp)
sw    s11,  48(sp)

# save stack pointer of previous task
sw    sp, 0(a0)
# switch to stack pointer of next task
mv    sp, a1

# Restore the next task's saved registers
lw    ra,    0(sp)
lw    s0,    4(sp)
lw    s1,    8(sp)
lw    s2,   12(sp)
lw    s3,   16(sp)
lw    s4,   20(sp)
lw    s5,   24(sp)
lw    s6,   28(sp)
lw    s7,   32(sp)
lw    s8,   36(sp)
lw    s9,   40(sp)

```

```

lw    s10, 44(sp)
lw    s11, 48(sp)

# restore pointer
addi sp, sp, 52
ret

```

2. In our round-robin scheduler, we initially updated the task by

```
next = (next + 1) % num_tasks;
```

However, the % operator can be inefficient in our small RISC-V core, especially when `num_tasks` is not a power of 2. Since we know that after incrementing, `next` is either equal to or less than `num_tasks`, we can instead use

```

next++;
if (next == num_tasks)
    next = 0;

```

So that we only need to check a single equality. This can be optimized even further by converting the entire function to RISC-V. Yield is initially implemented in C, with form

```

yield():
    disable_interrupts()
    pick_next()
    context_switch()

```

With this approach, the compiler may generate a normal stack frame for `yield()`, including prologue and epilogue code that saves registers. The `context_switch()` routine then creates its own saved context frame on the task stack, potentially saving the same architectural state a second time. To avoid this redundant stack-frame and register-save overhead, `yield()` is translated to RISC-V assembly, where task selection, context saving, stack-pointer switching, and context restoration are combined in one routine.

```

yield:
    # Disable interrupts
    csrci mstatus, 8

    # Pick the next task. Same logic as before
    la    t0, current_task
    lw    t1, 0(t0)
    la    t2, num_tasks
    lw    t2, 0(t2)
    addi  t3, t1, 1
    blt   t3, t2, 1f
    li    t3, 0
1:
    beq   t1, t3, .yield_done
    sw    t3, 0(t0)

    # Save the current task's callee-saved registers
    ...

    # Swap stack pointers
    la    t0, tasks
    slli  t4, t1, 3
    add   t4, t0, t4
    sw    sp, 0(t4)
    slli  t5, t3, 3
    add   t5, t0, t5
    lw    sp, 0(t5)

    # Restore the next task's saved registers
    ...

```

```
.yield_done:
    # Enable interrupts
    csrsi mstatus, 8
    ret
```

3.3 Prototype Evaluation Procedure

The prototype was evaluated in two complementary environments: QEMU and an FPGA-based RISC-V softcore. The purpose of using both environments was to separate functional validation from hardware-timing validation. QEMU was used first because it allowed rapid testing of the scheduler and task model in a controlled RISC-V environment before deploying to hardware. The FPGA implementation was then used to evaluate execution behavior under real processor, memory, interrupt, and UART conditions. This follows the project’s overall development path: first validating the link-training workload in simulation, then migrating it into C and integrating it with the scheduler, and finally deploying the combined scheduler and workload on a RISC-V softcore implemented on an FPGA.

3.3.1 QEMU Evaluation Procedure

The QEMU evaluation was used as the first stage of prototype validation and performance characterization. In this environment, the scheduler firmware was cross-compiled for a RISC-V target and executed using the QEMU `virt` platform. This allowed the team to test the scheduler in a controlled RISC-V environment before deploying it to the FPGA, while also collecting preliminary cycle-count measurements using the RISC-V mcycle counter.

The evaluation used a pair of representative firmware tasks designed to force real switching between independent execution contexts. This task structure exercised the main scheduler paths required by the SerDes link-training workload, including task creation, cooperative yielding, round-robin task selection, stack switching, and timer-driven preemption. By switching between two active tasks, the test verified that the scheduler was not simply

returning to the same execution context, but was actually saving and restoring task state correctly.

The procedure began with baseline measurements so that later timing results could be interpreted properly. These baselines included the overhead of reading the cycle counter, the cost of a simple memory access sequence, and the minimum observable loop delay during repeated cycle-counter reads. After this calibration step, the cooperative scheduling path was measured by repeatedly yielding between the two tasks and recording the minimum, maximum, and average round-trip cycle count. The low-level context-switch routine was then measured separately to isolate the cost of saving and restoring task state without including the full scheduling decision logic. Finally, the preemptive scheduling path was evaluated by arming the machine timer, enabling timer interrupts, and detecting the larger cycle gaps caused by interrupt entry, context saving, task selection, context switching, and interrupt return.

The QEMU results were used to evaluate both correctness and relative scheduler overhead. They confirmed that task creation, cooperative switching, direct context switching, and preemptive switching behaved as intended, while also providing early cycle-count estimates for the major scheduler operations. However, these measurements were interpreted as preliminary rather than final hardware timing results. In QEMU, the mcycle counter is closer to an emulated instruction-count-style measurement than a true representation of FPGA execution time, memory latency, or peripheral timing. For this reason, QEMU was useful for comparing scheduler paths and identifying high-overhead code, while the FPGA evaluation was still required to measure the scheduler under real softcore hardware conditions.

3.3.2 Limitations of QEMU Evaluation

A key limitation of the QEMU evaluation is that the emulated hardware does not match the final FPGA prototype. The QEMU setup uses the generic 32-bit RISC-V virt machine with a generic rv32 CPU model, rather than emulating the DE10-Lite board, the NEORV32 softcore, or the exact memory and peripheral system used in the FPGA implementation. As a result, QEMU is useful for evaluating the scheduler under a bare-metal RISC-V execution model, but it should not be treated as a cycle-accurate representation of the final hardware platform.

The firmware was compiled for an RV32IM_Zicsr target, meaning that the emulated processor supports the 32-bit integer instruction set, integer multiply/divide instructions, and control/status register access. This is sufficient for testing the scheduler’s core behavior, including task switching, machine-mode interrupt handling, timer setup, and cycle-counter reads. However, QEMU’s generic RISC-V CPU does not reproduce the exact instruction timing, memory latency, interrupt entry timing, or peripheral behavior of the FPGA softcore. Therefore, cycle counts collected in QEMU are best interpreted as deterministic virtual measurements, not final hardware timing results.

This limitation is especially important because the QEMU run configuration uses deterministic instruction counting through `-icount shift=0,sleep=off`. This makes the benchmark repeatable and useful for comparing the relative cost of different scheduler paths, since virtual time advances in a controlled way as instructions execute. However, this does not mean that one QEMU “cycle” corresponds exactly to one FPGA clock cycle. Real hardware performance depends on the softcore pipeline, memory implementation, bus timing, peripheral latency, and interrupt controller behavior, none of which are fully captured by the generic QEMU virt platform.

QEMU also provides only a simplified peripheral environment. The scheduler test uses emulated machine-mode CSRs, a CLINT-style timer, memory, and a UART console. These components are sufficient for testing timer-driven preemption and printing benchmark results, but they do not reproduce all hardware integration issues seen on the FPGA. For example, UART synchronization, reset behavior, timing interactions with external host communication, and FPGA-specific memory-mapped peripheral behavior must still be validated on the physical prototype.

For these reasons, QEMU was used as an early validation and characterization environment rather than the final source of performance evidence. It allowed the team to confirm that the scheduler behaved correctly, collect preliminary cycle-count data, and compare the relative overhead of cooperative switching, direct context switching, and preemptive switching. The FPGA evaluation was then required to determine whether the scheduler met timing expectations under actual softcore hardware conditions.

3.3.3 FPGA Evaluation Procedure

The FPGA evaluation was used as the hardware validation stage of the prototype. Unlike the QEMU environment, which executed the firmware on a generic virtual RISC-V platform, the FPGA implementation ran the scheduler on a physical RISC-V softcore. This allowed the team to evaluate the scheduler under more realistic processor, memory, interrupt, and peripheral conditions.

The scheduler firmware was compiled for the FPGA RISC-V target and loaded onto the softcore. The same core scheduler behaviours evaluated in QEMU were then tested on hardware: task creation, cooperative yielding, context switching, round-robin task selection, timer-driven preemption, and task-state preservation. This allowed the team to check whether the firmware remained correct after moving from the virtual environment to the FPGA implementation.

The timing evaluation followed the same general structure as the QEMU procedure. First, baseline measurements were collected to account for the overhead of reading the cycle counter and performing simple memory operations. These baselines provided context for interpreting scheduler timing results. The cooperative scheduling path was then measured by repeatedly yielding between two active tasks and recording the minimum, maximum, and average cycle counts. A lower-level context-switch test was also performed to isolate the cost of saving and restoring task state without including the full scheduling decision logic. Finally, the preemptive scheduling path was tested using the hardware timer interrupt, allowing the team to measure the overhead of interrupt entry, context saving, task selection, switching, and interrupt return.

In addition to these scheduler microbenchmarks, the FPGA was evaluated in a more application-representative SerDes training setup. In this test, the FPGA executed the firmware scheduler and equalization update tasks, while the host computer simulated the SerDes channel. Communication between the two systems occurred over UART. For each training step, the host provided the firmware with link-state information such as the current error and receiver history, and the FPGA returned updated equalizer parameters. This allowed the team to test whether the scheduler could support repeated equalization updates across scheduled tasks, rather than only switching between synthetic benchmark tasks.

The FPGA evaluation therefore served two purposes. First, it provided hardware-based cycle measurements for the scheduler’s main operations. Second, it demonstrated that the scheduler could be integrated into a simplified closed-loop link-training workflow. Together, these tests provided stronger evidence than QEMU alone that the scheduler could operate as part of a resource-constrained embedded SerDes firmware system.

3.3.4 Limitations of the FPGA Evaluation

Although the FPGA evaluation provided more realistic evidence than QEMU, it was still not equivalent to testing inside a commercial SerDes product. The FPGA prototype used a RISC-V softcore implemented on an FPGA, not the exact embedded controller, memory subsystem, clocking architecture, or peripheral environment used in a production SerDes device. As a result, the measured cycle counts are representative of this prototype platform, but they should not be interpreted as universal timing results for all RISC-V implementations or for Marvell’s final hardware environment.

The FPGA softcore also differs from a production microcontroller in its instruction timing, memory latency, bus architecture, interrupt handling, and clock frequency. These hardware details can significantly affect scheduler performance. This was observed directly in the prototype: simple load and store instructions were measured to take approximately 9 cycles each. Since context switching and interrupt handling require many register saves and restores, these memory-access costs had a large effect on the reported timing values. As a result, the measured cycle counts reflect both the scheduler implementation and the characteristics of the FPGA softcore memory system. The same assembly-level scheduler could perform differently on a processor with faster stack memory or lower bus latency.

Another limitation is that the SerDes channel itself was not implemented as a physical high-speed electrical link. Instead, the channel and receiver response were simulated on a host computer, while the FPGA handled the scheduler and equalization update logic. This was appropriate for a proof-of-concept because it allowed the team to test scheduler-controlled link training without requiring expensive SerDes measurement hardware. However, it means that the prototype did not capture all behaviours of a real SerDes link, such as analog front-end dynamics, clock and data recovery behaviour, true channel noise, jitter, process variation, temperature effects, or hardware settling times.

Finally, the FPGA benchmark workload was still a simplified representation of the final use case. The scheduler tests measured important operations such as cooperative switching, direct context switching, and timer-driven preemption, but they did not fully reproduce a complete multi-lane commercial SerDes system. A production system would include more lanes, more complex equalization stages, shared analog resources, asynchronous link events, failure recovery routines, and stricter coordination between training and normal operation. For this reason, the FPGA evaluation should be interpreted as proof-of-concept validation of the scheduler architecture rather than full production qualification.

3.4 Scheduler Performance Benchmarking

In this section we report the performance of our scheduler in terms of cycle counts in both QEMU and the FPGA test-bed using the procedures outlined in section 3.3.

3.4.1 QEMU Benchmarking

Table 5 reports the cycles required for various scheduler operations as well as load word and store word cycle counts for the QEMU architecture as a reference.

Table 5: QEMU Scheduler Benchmarking Results			
Function/Operation	Min Cycles	Max Cycles	Avg Cycles
Load Word			2
Store Word			2
Context Switch	38	46	38
Co-operative Yield	53	58	53
Task Creation	36	36	36
Pre-emptive Switch	Hardware interrupts are not enabled for QEMU benchmarking because of emulator specific hardware virtualization.		

The scheduler decision time is the difference between the context switch and cooperative yield cycle counts. From the above, on average, the decision time requires 15 cycles. All operations fall within our constraints from table 1. However, note that these counts are optimistic due to the limitations identified in section 3.3.2. Now we look at benchmarking on the FPGA platform.

3.4.2 FPGA Benchmarking

We present the raw FPGA scheduler benchmarking results in table 6. Once again, these results are overly pessimistic because of the limitations that we outlined in section 3.3.4. We can see this from the load word and store word operations requiring 9 cycles each.

Table 6: FPGA Scheduler Benchmarking Results			
Function/Operation	Min Cycles	Max Cycles	Avg Cycles
Load Word			9
Store Word			9
Context Switch	190	219	190
Co-operative Yield	243	258	243
Task Creation	344	344	344
Pre-emptive Switch	381	381	381

For a more fair assessment, we also provide a memory-normalized estimate of the scheduler results. The raw FPGA cycle counts are affected significantly by the memory system of the FPGA softcore. In particular, simple load-word and store-word operations were measured to require approximately 9 cycles each. This has a large effect on scheduler operations because context switching, cooperative yielding, task creation, and pre-emptive switching all rely heavily on saving and restoring register state to memory.

To estimate how the scheduler would perform on a lower-latency embedded memory system, the raw FPGA measurements were scaled by a factor of $2/9$. This factor corresponds to replacing the measured 9-cycle load/store latency with an assumed 2-cycle load/store latency, which is more representative of a tightly coupled SRAM or lower-latency embedded memory implementation. This scaling is not intended to be a precise hardware prediction, since not every instruction in the scheduler is a load or store. Branches, CSR accesses, timer operations, and scheduling-decision logic would not necessarily scale by the same amount. However, because the dominant cost in context switching and interrupt handling is register save/restore activity, this scaling provides a useful first-order estimate of how much of the measured overhead is caused by the FPGA memory system rather than by the scheduler structure itself.

Therefore, the scaled values should be interpreted as an approximate memory-normalized comparison, while the unscaled values remain the actual measured FPGA performance. This distinction is important because the raw FPGA results characterize the prototype platform, whereas the scaled results better indicate the expected scheduler behaviour on an embedded implementation with faster memory access. We provide the scaled values below:

Table 6: FPGA Scheduler Benchmarking Results (Scaled by 2/9)			
Function/Operation	Min Cycles	Max Cycles	Avg Cycles
Context Switch	42	48	42
Co-operative Yield	54	57	54
Task Creation	76	76	76
Pre-emptive Switch	85	85	85

After applying this memory-normalized scaling, the scheduler satisfies the project constraints for context switching, cooperative yielding, and task creation. The only remaining constraint not met is the pre-emptive switching requirement. This result is expected because a true pre-emptive switch is inherently more expensive than a cooperative yield. In a cooperative switch, the task yields at a known safe point, so the scheduler only needs to preserve the reduced context required by the calling convention. In contrast, a pre-emptive interrupt can occur at any point during task execution, so the interrupt handler must conservatively save the full active processor state before switching tasks. This includes substantially more register-save and register-restore operations, as well as interrupt-specific control state. As a result, even with lower-latency memory, the cost of a fully pre-emptive switch is unlikely to meet the original cycle target on this type of implementation. However, this does not invalidate the selected scheduler design. In later discussions, Marvell indicated that cooperative scheduling was sufficient for the intended SerDes training workload, since link-training routines can be structured around explicit yield points. Therefore, the prototype's most relevant result is that the cooperative scheduler path satisfies the timing requirements while preserving flexibility for programmable SerDes training tasks.

3.5 Evaluating Scheduler for SerDes Application

The C-based SerDes equalization and lane training testbench discussed in section 3.2.2 was combined with the C-based firmware scheduler implementation running on the FPGA softcore implementation to obtain a comprehensive testing environment for the scheduler using SerDes tasks.

A Python-based graphical interface was developed to enable real-time interaction between the FPGA-based firmware scheduler and a software-defined SerDes link model. This interface functions both as a visualization tool and as a communication bridge, supporting closed-loop adaptation of equalization parameters during link training.

3.5.1 System Overview

The application implements a bidirectional communication pipeline over a UART interface using a USB-to-serial bridge (CP2102). The FPGA executes the link training algorithm and produces updated equalization coefficients, while the Python application emulates the forward channel, computes error metrics, and returns feedback to the FPGA. This partitioning allows computationally intensive signal processing and visualization to be performed on the host PC, while the FPGA handles latency-sensitive equalizer updates.

The software is organized into three main components: a channel simulation module, a serial communication worker, and a graphical user interface.

3.5.2 Channel Simulation

The forward SerDes link is modeled in software to emulate realistic transmission conditions. A pseudo-random binary sequence (PRBS) is generated and upsampled according to a fixed oversampling factor. The transmit waveform is optionally processed by a feed-forward equalizer (FFE), and then passed through a channel modeled as a finite impulse response (FIR) filter, with coefficients loaded from an external file.

The received waveform is quantized using a low-resolution analog-to-digital converter (ADC) model to approximate receiver front-end constraints. A simple clock and data recovery (CDR) algorithm selects the optimal sampling phase by maximizing signal

magnitude across candidate phases. The resulting sampled signal forms the input to the equalization loop.

3.5.3 Serial Communication Protocol

Communication between the PC and FPGA is implemented using a custom binary protocol over UART, enabling deterministic and low-overhead data exchange. Each iteration of the training loop consists of two packet transfers:

- **PC → FPGA (Error Packet):** Contains a synchronization byte, lane identifier, command field, scalar error value, and the current receive buffer and decision history. This packet provides the FPGA with sufficient context to update equalizer coefficients.
- **FPGA → PC (Tap Packet):** Contains a synchronization byte, lane identifier, status flag, and updated equalization coefficients (FFE and DFE taps). The synchronization byte ensures packet alignment, while the status flag indicates events such as lane resets.

A dedicated background thread handles all serial communication to prevent blocking the graphical interface. This thread maintains per-lane state, enforces the request–response ordering of packets, and ensures consistent synchronization between the PC and FPGA.

3.5.4 Multi-Lane Training and Scheduling

The system supports multiple independent “lanes,” each corresponding to a separate instance of the link training algorithm. The Python application processes all lanes sequentially for each received sample, maintaining independent buffers, equalizer states, and performance metrics.

For each lane, the equalized output is computed using the current FFE and DFE coefficients. A hard decision is then applied to estimate the transmitted symbol, and the error between the estimated and desired symbol is calculated. This error is accumulated to compute the mean squared error (MSE), which is periodically reported for visualization.

The FPGA scheduler updates the equalization coefficients based on the received error signal, effectively time-multiplexing the training process across multiple lanes. This approach

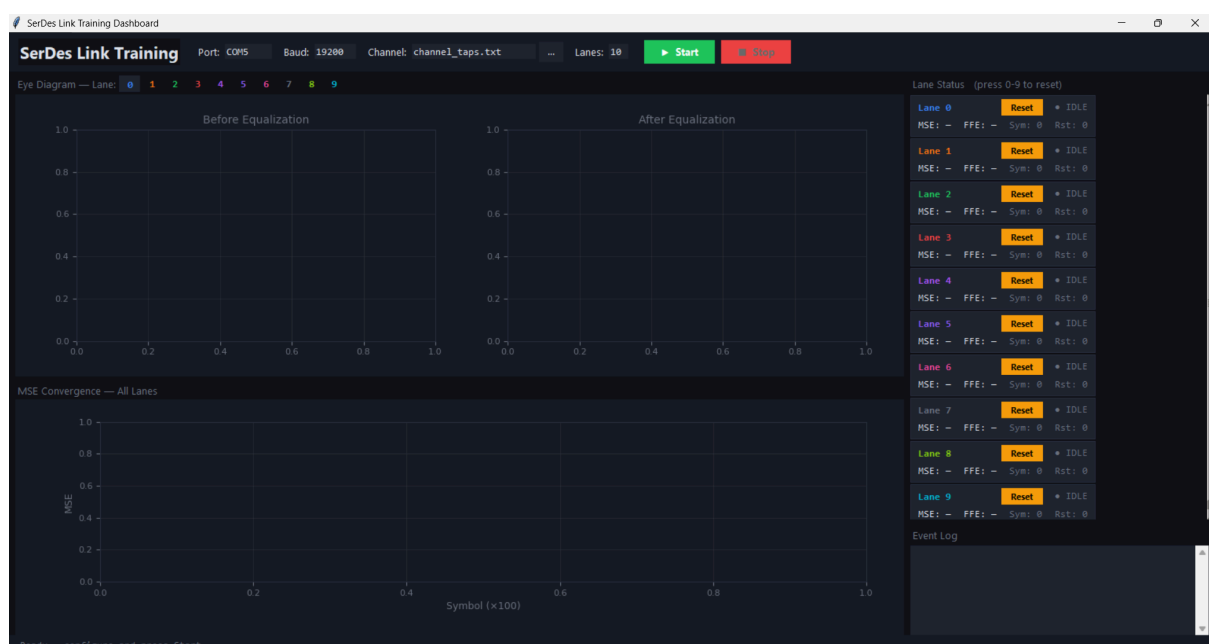
enables concurrent evaluation of multiple training instances while maintaining a single shared hardware implementation.

3.5.5 Graphical User Interface

The graphical interface, shown in Figure X, implemented using Tkinter and Matplotlib, provides real-time insight into link performance and convergence behavior. Key features include:

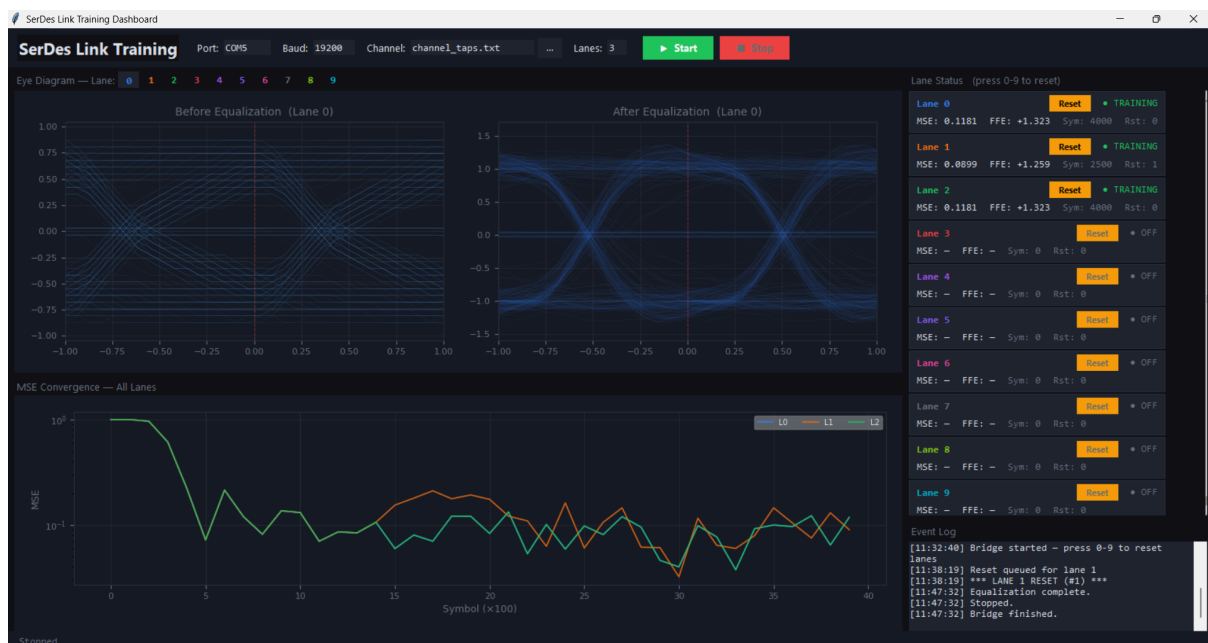
- **Eye Diagrams:** Displayed before and after equalization for a selected lane, enabling visual assessment of signal integrity and equalizer effectiveness.
- **MSE Convergence Plots:** Logarithmic plots of MSE versus time for all lanes, illustrating convergence rates and stability.
- **Per-Lane Status Dashboard:** Displays key metrics including current MSE, processed symbol count, main FFE tap value, and reset count. Individual lanes can be reset via keyboard input or GUI controls.
- **Event Logging:** Provides timestamped feedback on system events such as resets, communication errors, and initialization status.

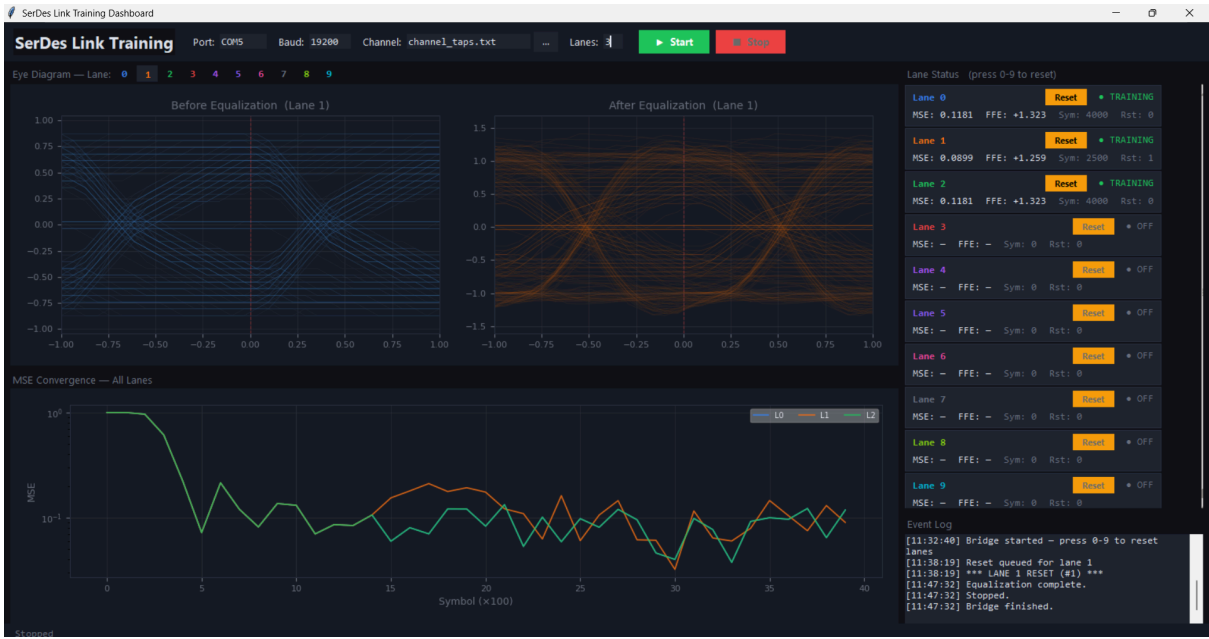
The GUI operates asynchronously, periodically polling a thread-safe queue populated by the communication worker. This architecture ensures responsive updates without interfering with real-time data exchange.



3.5.6 PC-Based SerDes Link Training Interface Example Operation

Figures X and Y depict the GUI window once training for all lanes has been completed for a sample operation sequence. In this run, a soft reset was performed for lane 1 at symbol 1400, resulting in a deviation between the MSE training curves. Furthermore, the eye diagram for lane 1, shown in figure Y is slightly degraded with respect to the diagram in figure X since the training algorithm was restarted partway during the PRBS. However, the top-right and bottom-left panels of the GUI reveal that the MSE values eventually converge despite the soft reset and additional logging proved that the final tap values also converged sufficiently. The predictable steady-state behavior of this training sequence, verified across different lane counts suggests efficient and predictable operation of the scheduler across varying operating conditions.





Chapter 4. Discussion Conclusions and Future Considerations

TODO

Chapter 5. Project Team

Satvick Acharya

Satvick Acharya is an Engineering Science student at the University of Toronto specializing in Electrical and Computer Engineering, with a strong focus on embedded systems, automotive software, and system-level design. Raised in the Greater Toronto Area, Satvick brings a systems-oriented and globally informed perspective shaped by years of involvement in Model United Nations, where he developed a deep appreciation for complex, multi-stakeholder systems and structured decision-making under constraints.

Satvick has industry experience as an Embedded Automotive Software Engineer Intern at Qualcomm, where he worked on safety-critical video drivers and CI infrastructure for a dual-chiplet automotive SoC. His work emphasized correctness, low latency, and system reliability in real-world deployment. He has also contributed to autonomous vehicle research and competitions through the ESROP fellowship and the aUToronto self-driving car team, leading efforts in deep learning acceleration and real-time perception pipelines.

Complementing his software background, Satvick is completing an undergraduate thesis under Professor Ali Sheikholeslami on a PAM3 simultaneous bidirectional transceiver for die-to-die links, giving him circuit-level insight into high-speed interconnect design.

Satvick offered theoretical support during the development of the initial MATLAB-based SerDes equalization and link training testbench. He led the C-based scheduler implementation in QEMU and on the FPGA. Satvick contributed to the evaluation of the scheduler for SerDes application by developing the PC-based SerDes link training interface, and technical writing for deliverables.

Matthew Oliveira

Matthew Honorio Oliveira is a fourth-year Engineering Physics undergraduate at the University of Toronto with research interests in error correction for reliable wireline and wireless data transmission. With a cross-disciplinary foundation spanning physics, mathematics, and engineering, he enjoys translating rigorous ideas from coding and

information theory into practical, system-level design decisions, especially where robustness, performance limits, and verification discipline are essential.

Matthew gained industry experience at Advanced Micro Devices (AMD), working with wireline SERDES and contributing to implementation and validation efforts that support PCIe Gen 6-class high-speed I/O in advanced designs. He has also pursued international research at RWTH Aachen University through the Institute for Quantum Information (IQI), studying quantum error correction and information theory in a collaborative, cross-cultural environment.

Outside of work, Matthew enjoys puzzles, whether in games or real-world systems, and brings that same structured, curiosity-driven approach to teamwork and problem solving.

Gustavo Nakashita

Gustavo Nakashita is a 4th year Undergraduate student at the University of Toronto in Engineering Science specializing in Electrical and Computer Engineering. He was born in Brazil but moved to and was raised in Singapore, where he experienced a wealth of cultural diversity from all continents and all parts of the world, growing a deep respect for the various ways of life of many people. He is fluent in English and Portuguese.

Gustavo has strong hobbies in optimizing gameplay for performance in video games, such as in Chess where openings and tactical ideas are necessary to gain the upper hand, or in Roguelikes where each decision has drawbacks and advantages that must be weighed carefully throughout an entire playthrough. These hobbies play a role in Engineering work, as many problems have multiple solutions with their own tradeoffs, and weighing these between each other is important, especially when considering the scope of a project.

Gustavo brings strong foundations in programming, having TA'd an introductory programming course in Python and C for Engineering students. Additionally, during university, he has explored various fields in ECE, including signal processing and communication systems, computer architecture, and digital systems design. He has also performed summer research developing an experimental control system in MatLab for

ultrafast electron diffusion studies, showcasing an ability to apply these skills in a Physics research setting.

Gustavo assisted with the development of the MatLab wireline model and the C scheduler system, including the customized programmable scheduler. Gustavo was also responsible for taking notes and organizing documents regarding meetings and discussions, as well as establishing communication with Marvell and Shiekholeslami to discuss meeting minutes and additional questions outside meetings. Gustavo also contributed to figures and writing in the team deliverables.

Andreas Conostas-Malevanets

Andreas is a 4th-year Engineering Science student specializing in engineering physics. He is focused on the design of analog circuits for high-speed wireline and optical communication. He is most interested in the development of analytical frameworks and architectures within this field.

Andreas has worked in analog design during his 16th-month industry co-op and has continued to do so in industry and for his thesis. He has also had previous experience in high-speed printed-circuit board design through research and coursework, and has further worked on equalization during his co-op, providing him with a broader view of the considerations for a high-speed chip outside the analog blocks themselves. He has furthermore previously worked on embedded systems for cardiopulmonary resuscitation systems in a research setting. Andreas completed his undergraduate thesis in the design of a high-swing transimpedance amplifier.

Andreas led the development of a MATLAB model for the transceiver system and its equalization, and provided theoretical support on SerDes throughout the project. He contributed to the C-based scheduler implementation and technical writing for deliverables.

References

- [X1] E. C. de Lima, F. D. Rossi, M. C. Luizelli, R. N. Calheiros, and A. F. Lorenzon, “A neural network framework for optimizing parallel computing in cloud servers,” *Journal of Systems Architecture*, vol. 150, p. 103131, 2024, doi: 10.1016/j.sysarc.2024.103131.
- [X2] PCI-SIG, “PCI Express 6.0 Specification,” PCI-SIG, Beaverton, OR, USA. Accessed: Mar. 7, 2026.
- [X3] J. D’Ambrosia, “Ethernet’s Next Bar Is Now – 800 Gb/s!,” IEEE Standards Association, 2024. [Online]. Available: <https://standards.ieee.org/beyond-standards/ethernets-next-bar/>
- [X4] H. Shakiba, D. Tonietto, and A. Sheikholeslami, “High-Speed Wireline Links—Part I: Modeling,” *IEEE Open Journal of the Solid-State Circuits Society*, vol. 4, pp. 97–109, 2024, doi: 10.1109/OJSSCS.2024.3433324.
- [X5] Synopsys, “What is SerDes?,” Synopsys, 2026. [Online]. Available: <https://www.synopsys.com/glossary/what-is-serdes.html>
- [X6] W. Chen, “SerDes Architectures and Applications,” *DesignCon*, 2004. [Online]. Available: <https://chenweixiang.github.io/2017/08/27/serdes.html>
- [X7] Advanced Micro Devices, Inc., *7 Series FPGAs GTX/GTH Transceivers User Guide (UG476)*, v1.12.1, Aug. 14, 2018. [Online]. Available: https://docs.amd.com/v/u/en-US/ug476_7Series_Transceivers
- [X8] Intel Corporation, *L- and H-Tile Transceiver PHY User Guide*, document ID 683621, Feb. 28, 2025. [Online]. Available: <https://www.intel.com/programmable/technical-pdfs/683621.pdf>
- [X9] ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva, Switzerland: ISO/IEC/IEEE, Nov. 2018.
- [X10] M. Verma, “Tightly-Coupled Analog and DSP Architecture for Best 112G SerDes IP Performance,” *Synopsys IP Technical Bulletin*, Apr. 19, 2021. [Online]. Available: <https://www.synopsys.com/articles/best-112g-serdes-ip-performance.html>
- [X11] RISC-V International, “FAQ,” *RISC-V International*, accessed Feb. 11, 2026. [Online]. Available: <https://riscv.org/about/faq/>

- [X12] R. Rocher and M. Sawan, “Accuracy evaluation of fixed-point based LMS algorithm,” *Digital Signal Processing*, vol. 20, no. 3, pp. 640–652, May 2010, doi: 10.1016/j.dsp.2009.10.007.
- [X13] Analog Devices, “Fixed-Point vs. Floating-Point Digital Signal Processing,” *Analog Devices Technical Articles*, Mar. 18, 2021. [Online]. Available: <https://www.analog.com/en/resources/technical-articles/2021/03/18/19/16/fixedpoint-vs-floatingpoint-dsp.html>
- [X14] FreeRTOS, “FreeRTOS FAQ - Memory usage, Boot Times & Context Switch Times,” FreeRTOS, <https://www.freertos.org/Why-FreeRTOS/FAQs/Memory-usage-boot-times-context> (accessed Mar. 11, 2026).
- [X15] NXP Semiconductors, “Measuring interrupt latency,” NXP Semiconductors, <https://www.nxp.com/docs/en/application-note/AN12078.pdf> (accessed Mar. 12, 2026).
- [X16] NXP Community, “Cycles per instruction?,” *NXP Forums*, Jun. 15, 2016. [Online]. Available: https://community.nxp.com/t5/LPCXpresso-IDE/Cycles-per-instruction/td-p/545211?utm_source=chatgpt.com
- [X17] B. Miller, F. Vahid, and T. Givargis, “RIOS: A Lightweight Task Scheduler for Embedded Systems,” *Proceedings of the Workshop on Embedded and Cyber-Physical Systems Education*, pp. 1–7, Oct. 2012. doi:10.1145/2530544.2530553
- [X18] P. Levis et al., “TinyOS: An operating system for Sensor Networks,” *Ambient Intelligence*, pp. 115–148, 2005. doi:10.1007/3-540-27139-2_7
- [X19] R. Balas, A. Ottaviano, and L. Benini, “CV32RT: Enabling fast interrupt and context switching for RISC-V microcontrollers,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 6, pp. 1032–1044, Jun. 2024. doi:10.1109/tvlsi.2024.3377130
- [X20] J. Nutini, I. Laradji, and M. Schmidt, “Let’s Make Block Coordinate Descent Converge Faster: Faster Greedy Rules, Message-Passing, Active-Set Complexity, and Superlinear Convergence,” *arXiv*, arXiv:1712.08859, Dec. 2017. doi: 10.48550/arXiv.1712.08859.
- [X21] R. Yang, Z. Dong, P. Cheng, Y. Xiu, Y. Li, and N. Wei, “Joint Transceiver Design for RIS Enhanced Dual-Functional Radar-Communication with Movable Antenna,” *arXiv*, arXiv:2502.09023, Feb. 2025. doi: 10.48550/arXiv.2502.09023.
- [X22] J. T. Humphries, K. Kaffes, D. Mazières, and C. Kozyrakis, “A case against (most) context switches,” *Proceedings of the Workshop on Hot Topics in Operating Systems*, pp. 17–25, Jun. 2021. doi:10.1145/3458336.3465274

- [X23] FreeRTOS , “FreeRTOS Documentation,” FreeRTOS ,
<https://www.freertos.org/Documentation/00-Overview> (accessed Mar. 11, 2026).
- [X24]

Appendix A: Standard Equalization Techniques

Technical Introduction to SerDes Equalization

Modern high-speed wireline communication systems operate at data rates exceeding 200 Gb/s. At these speeds, conventional conductors behave like insulators, and signal integrity is significantly degraded by channel loss and reflections, alongside jitter, noise, and gain compression. Figure 12 shows the frequency response of a typical 4-port channel transmitting from port 1 to port 2 (i.e., the signal attenuation for a signal moving across a wire as a function of frequency) [SHEIK].

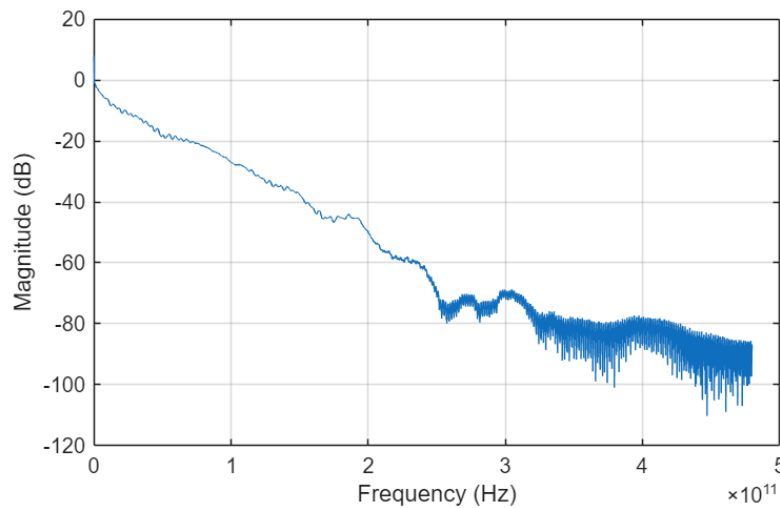


Figure X: Frequency response of a wireline channel for transmitted signals

From Figure X, assuming each symbol contains 1 bit of information at 200 Gb/s, we would expect an attenuation of ~ 50 dB relative to the input signal. To maintain reliable communication, state-of-the-art SerDes transceivers modify the signal in a way that inverts the effects of channel loss. To this end, SerDes transceivers employ a combination of transmitter and receiver equalization techniques, including feed-forward equalization (FFE), continuous-time linear equalization (CTLE), and decision feedback equalization (DFE) to produce a readable signal at the receiver end, in a manner similar to the design presented in Figure Y.

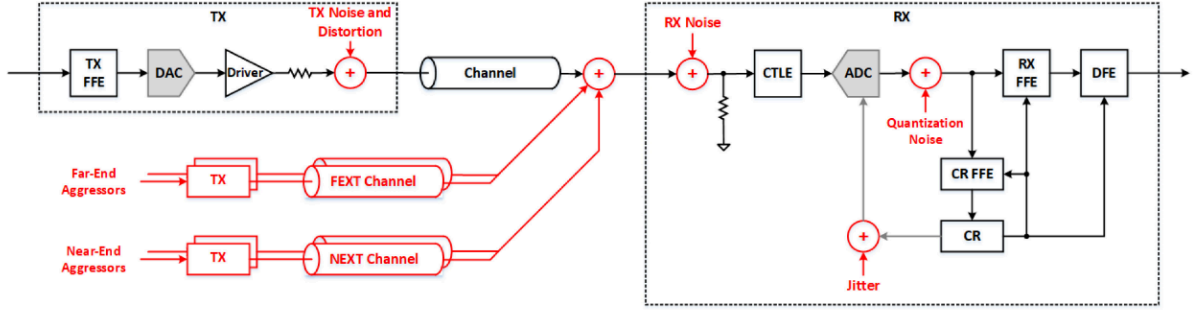


Figure Y: Block diagram of a wireline transceiver. Noise, jitter, and crosstalks are shown in red. Reprinted from [SHEIK]

Feed-Forward Equalization (FFE)

FFE is a linear equalization technique which combats intersymbol interference (ISI) by filtering the transmitted or received waveform with a finite impulse response (FIR) filter. The key idea is to reshape the signal so that, after passing through the channel, the overall frequency response of the combined FFE plus the channel is flat over the signal bandwidth. Mathematically, the FFE can be modeled as a tapped-delay line:

$$y[n] = \sum_{k=0}^{N-1} w_k x[n - k] \quad (1)$$

where $x[n]$ is the input sample stream, w_k are the tap coefficients, and $y[n]$ is the equalized output. In the frequency domain, FFE can be expressed as:

$$Y[f] = X[f] \sum_{k=0}^{N-1} w_k e^{-j2\pi f k} = X[f] W[f] \quad (2)$$

where we design $W[f]$ to approximate $1/H[f]$, where $H[f]$ is the channel response. FFE is done both at the receiver (RX) with post-cursor components (i.e., taps after the main symbol $x[n]$) and again at the transmitter for post-cursor and precursor components [SHEIK]. The FFE as a linear equalizer is limited in its ability to correct non-linear distortions, and also boosts the noise content of the received signal.

Continuous-time linear equalization CTLE)

CTLE is an analog, continuous-time equalization technique that mitigates ISI by boosting attenuated, high-frequency components of the received waveform prior to quantization. Because wireline channels behave like a low-pass filter (see figure 12), CTLE boosts high frequency components to flatten the combined frequency response of the channel plus equalizer. A common behavioural model for CTLE is a rational transfer function with a zero and one or more poles. CTLE can be implemented as a passive RC network or an active equalizer if amplification is needed. Since the CTLE boosts the high frequency components of a signal, it also boosts high frequency noise. Unlike the FFE, it is placed before the ADC so does not boost quantization noise, as this noise has yet to be introduced. However, since it requires that a transfer function be synthesized in analog, the number of weights that can be controlled is fewer.

Decision Feedback Equalization (DFE)

DFE is a nonlinear receiver equalization technique that cancels post-cursor ISI by using previous symbol decisions to predict the interference they cause on the current symbol, and then subtracting that interference from the current symbol before inference. If the previous k -th symbol was inferred to be $\hat{d}[n - k]$, $r[n]$ is the current quantized signal and $z[n]$ is the output signal post-DFE, we may write

$$z[n] = r[n] - \sum_{k=1}^K b_k \hat{d}[n - k] \quad (3)$$

The tap coefficients b_k are chosen to approximate post-cursor ISI contributions of previously detected symbols. The advantage of DFE is that it can eliminate post-cursor ISI without boosting high-frequency noise. However, it requires an analog summer so it can be expensive. Therefore, it is often limited to a single tap.

Algorithms for Equalization Training

Least-mean-squares is the method of choice for training equalization taps. A pseudo-random bit sequence (PRBS) whose true values at each time step are known both to the transmitter

and receiver is sent over the channel, and for each symbol, the algorithm calculates the difference between the signal read at the receiver and the transmitted signal after the RX FFE and then after the DFE is applied. The goal of the algorithm is to minimize this error using gradient descent, iteratively changing the taps such that the difference between the transmitted and received signals is minimized.

Least-mean-squares (LMS) is widely used for adaptive equalization in communication systems because it provides an effective balance between convergence performance and implementation complexity. Unlike more computationally intensive algorithms such as recursive least squares (RLS), LMS requires only simple multiplications and additions per iteration, making it well suited for hardware and firmware implementations in high-speed serial links such as SerDes receivers. This low complexity allows the algorithm to run continuously during link training while maintaining stable convergence behavior even in noisy channels. For these reasons, LMS has become the standard adaptive algorithm used in many practical equalization systems, particularly in high-speed digital communication receivers where real-time coefficient adaptation is required [11].